

UNIVERSIDAD MAYOR DE SAN SIMÓN
FACULTAD DE CIENCIAS Y TECNOLOGÍA
INGENIERÍA EN SISTEMAS

CIENCIA DE DATOS Y MACHINE LEARNING

INFORME FINAL

Aprendizaje supervisado sobre covid_19_data.csv y Customer.csv:
modelos regresionales, de decisión y una red neuronal artificial
(perceptrón multicapa feedforward con backpropagation)

GESTIÓN: II / 2026

INTEGRANTES:

Cordova Nigoevic Anelis Melani
Cosio Marañon Luciana
Monje Lucas Rodrigo Andres
Moscoso Pinto Gabriel Alejandro
Vargas Soliz Lucas Brandon

GRUPO: 5

DOCENTE: Ing. Patricia Erika Rodriguez Bilbao

FECHA: 29/09/26

COCHABAMBA – BOLIVIA

ÍNDICE

I. Introducción.....	3
II. Objetivos.....	4
2.1. Objetivo general.....	4
2.2. Objetivos específicos.....	4
III. Marco teórico.....	5
3.1. Aprendizaje supervisado.....	5
3.2. Pre-procesamiento de datos.....	5
3.3. Áreas y modelos.....	6
3.3.1. Modelos regresionales.....	6
3.3.2. Modelos de decisión.....	6
3.3.3. Redes neuronales artificiales.....	6
3.4. Métricas de evaluación.....	8
3.5. Justificación de la elección de modelos.....	8
IV. Ingeniería.....	10
4.1. Descripción, ampliación y abstracción del problema.....	10
4.1.1. Problema 1: fallecidos por COVID-19 (regresión).....	10
4.1.2. Problema 2: segmento comercial de un cliente (clasificación).....	10
4.2. Tecnologías utilizadas.....	11
4.3. Arquitectura del sistema.....	12
4.4. Pre-procesamiento de datos.....	13
4.4.1. Proceso 1: incompletitud y técnica aplicada a los datos faltantes.....	13
4.4.2. Proceso 2: ruido y excepciones.....	13
4.4.3. Proceso 3: conversión de datos.....	15
4.4.4. Estandarización.....	15
4.4.5. Correcciones al código del primer avance.....	16
4.5. Determinación de los datos de entrenamiento.....	17
4.5.1. Variables predictoras y objetivo.....	17
4.5.2. División en entrenamiento, validación y prueba.....	18
4.6. Modelos implementados.....	19
4.6.1. Ocho modelos de referencia.....	19
4.6.2. Red neuronal artificial desde cero.....	19
4.7. Validación con cinco métricas.....	24
4.7.1. Métricas elegidas y su justificación.....	24
4.7.2. Resultados en covid_19_data.csv.....	24
4.7.3. Resultados en Customer.csv.....	27
4.7.4. Control del sobreajuste.....	28
4.7.5. Análisis detallado de la red neuronal.....	30
4.7.6. Diagnóstico de Customer.csv: ¿hay señal que aprender?.....	32
4.7.7. Mejor ajuste sin sobreajuste: modelo elegido.....	33

4.8. Predicciones con datos ingresados por el usuario.....	34
4.8.1. Interfaz de predicción.....	34
4.8.2. Casos de prueba e interpretación: fallecidos por COVID-19.....	36
4.8.3. Casos de prueba e interpretación: segmento de un cliente.....	38
4.9. Aplicación integrada.....	38
V. Conclusiones.....	42
Limitaciones y trabajo futuro.....	42
VI. Bibliografía.....	44
Anexo. Estructura del proyecto y ejecución.....	45

I. Introducción

El desarrollo de un modelo de Machine Learning no empieza en el algoritmo, sino en los datos: su calidad, su preparación y la pregunta que se les hace determinan lo que cualquier modelo puede llegar a aprender. Este informe reúne en un solo documento el trabajo de las tres etapas del proyecto de la materia sobre dos conjuntos de datos asignados: `covid_19_data.csv`, con los registros diarios acumulados de casos, recuperados y fallecidos por COVID-19 por país y provincia, y `Customer.csv`, con la edad, la ubicación y el segmento comercial de 793 clientes.

En el **primer avance** se ejecutó el pre-procesamiento de ambos conjuntos: tratamiento de la incompletitud, eliminación de ruido y excepciones, y conversión de las variables de texto a una representación numérica. En el **segundo avance** se implementaron y compararon ocho modelos supervisados de cuatro familias —regresiones, árboles de decisión, máquinas de vectores de soporte y redes neuronales— para dos tareas: la **regresión** del número acumulado de fallecidos por COVID-19 y la **clasificación** del segmento comercial de un cliente.

Esta **entrega final** atiende las observaciones recibidas y completa los requisitos de la práctica. Se justifica cada decisión de ingeniería —técnicas de limpieza, elección de variables, partición de los datos, modelos y métricas—; se implementa **desde cero** una red neuronal artificial de tipo perceptrón multicapa, con propagación hacia adelante (*feedforward*) y aprendizaje por retropropagación del error (*backpropagation*); se valida cada modelo con **al menos cinco métricas** controlando el sobreajuste; y se construye una aplicación que permite **realizar predicciones con datos ingresados por el usuario** e interpretarlas. Todo el sistema —entrenamiento, resultados y predicción— se ejecuta con un solo comando.

Los resultados muestran dos situaciones opuestas, ambas útiles. En `covid` los modelos aprenden una relación fuerte: el mejor alcanza un R^2 de 0,9686 y la red neuronal propia 0,9632, con una diferencia entre entrenamiento y prueba de solo 0,0051. En `Customer.csv` ningún modelo supera el baseline de responder siempre la clase mayoritaria (0,5097); el informe demuestra con pruebas diagnósticas que la causa es la ausencia de relación entre las variables disponibles y el segmento, y no una falla de los modelos.

II. Objetivos

2.1. Objetivo general

Implementar, validar y comparar modelos de aprendizaje supervisado —con énfasis en una red neuronal artificial de tipo perceptrón multicapa entrenada con backpropagation— sobre los conjuntos `covid_19_data.csv` y `Customer.csv`, justificando cada decisión de ingeniería y poniendo los modelos a disposición del usuario para realizar predicciones con sus propios datos.

2.2. Objetivos específicos

- Pre-procesar ambos conjuntos con técnicas implementadas en código propio —incompletitud, ruido y excepciones, conversión— sin herramientas externas, justificando la técnica aplicada a los datos faltantes.
- Determinar y justificar las variables predictoras y la variable objetivo de cada problema, y particionar los datos de forma reproducible en entrenamiento, validación y prueba.
- Implementar ocho modelos de referencia de cuatro familias (regresión, árboles, vectores de soporte y redes neuronales) como línea base comparativa.
- Implementar desde cero, con NumPy, un perceptrón multicapa feedforward entrenado con backpropagation y descenso de gradiente con momento, y verificar numéricamente su gradiente.
- Validar cada modelo con al menos cinco métricas, compararlo contra un baseline trivial y controlar el sobreajuste mediante un conjunto de validación y parada temprana (*early stopping*).
- Realizar predicciones con datos ingresados por el usuario e interpretar los resultados.
- Integrar entrenamiento, resultados y predicción en una aplicación que se ejecute con un solo comando.

III. Marco teórico

3.1. Aprendizaje supervisado

El aprendizaje supervisado construye una función f que relaciona variables de entrada X con una variable objetivo y conocida, a partir de ejemplos etiquetados. Si y es una categoría, el problema es de **clasificación**; si es un valor numérico continuo, de **regresión**. Lo que interesa de un modelo no es cuánto acierta sobre los datos con los que aprendió, sino su capacidad de **generalización**: su desempeño sobre datos que nunca vio.

Un modelo con capacidad excesiva puede memorizar los ejemplos de entrenamiento, incluido su ruido, y fallar con datos nuevos: esto es el **sobreajuste** (*overfitting*). Uno con capacidad insuficiente no captura la relación real: **subajuste**. Para detectarlos, los datos se dividen en un conjunto de **entrenamiento**, con el que el modelo aprende; uno de **validación**, con el que se toman decisiones durante el desarrollo (hiperparámetros, cuándo detener el entrenamiento); y uno de **prueba**, que solo se usa al final para medir la generalización sin sesgo.

3.2. Pre-procesamiento de datos

La calidad de un modelo está acotada por la calidad de sus datos. El pre-procesamiento se organiza en tres procesos:

- **Incompletitud.** Los valores faltantes se pueden tratar por **eliminación** (de filas o de columnas completas) o por **imputación**: reemplazo por la media o la mediana (variables numéricas), por la moda (categóricas), por una constante, o por métodos basados en modelos como k vecinos más cercanos. La imputación es adecuada cuando el valor existe pero no se registró; si la ausencia es estructural —el dato no aplica a ese registro—, imputar inventa información.
- **Ruido y excepciones.** Incluye registros **duplicados**, que sobreponderan ciertos ejemplos, y valores **atípicos**. El método del **rango intercuartílico** (IQR) marca como atípico todo valor fuera de $[Q1 - 1,5 \cdot IQR, Q3 + 1,5 \cdot IQR]$, con $IQR = Q3 - Q1$. En variables de conteo con cola larga, aplicar antes la transformación $\log(1 + x)$ reduce la asimetría y evita que el IQR descarte los valores grandes legítimos.
- **Conversión.** Los algoritmos operan sobre números. Las variables categóricas nominales se convierten con **codificación one-hot**: una columna binaria por categoría (menos una, para evitar redundancia). Además, la **estandarización** $z = (x - \mu)/\sigma$ lleva cada variable a

media 0 y desviación 1; es indispensable para los modelos basados en distancias o gradientes (SVM, redes neuronales, modelos lineales regularizados).

3.3. Áreas y modelos

La práctica define tres áreas: modelos regresionales, modelos de decisión y redes neuronales artificiales. El proyecto implementa modelos de las tres, porque compararlas sobre los mismos datos es lo que permite justificar cuál conviene a cada problema.

3.3.1. Modelos regresionales

La **regresión lineal** modela el objetivo como una combinación lineal de las variables, $\hat{y} = w \cdot x + b$, y ajusta los pesos minimizando el error cuadrático. La **regresión Ridge** añade una penalización $\lambda \|w\|^2$ (regularización L2) que reduce los pesos grandes y con ellos la varianza del modelo. Para clasificación, la **regresión logística** aplica a la combinación lineal la función sigmoide (dos clases) o **softmax** (varias clases), y devuelve una probabilidad por clase.

3.3.2. Modelos de decisión

Un **árbol de decisión** divide recursivamente el espacio de variables con preguntas del tipo «¿ $x \leq c$?», eligiendo en cada paso el corte que más reduce la impureza (clasificación) o la varianza (regresión); la predicción es constante dentro de cada hoja. Es interpretable, pero un árbol profundo memoriza. **Random Forest** entrena muchos árboles sobre muestras aleatorias de filas y de columnas (*bagging*) y promedia sus predicciones, lo que reduce la varianza sin aumentar el sesgo.

Las **máquinas de vectores de soporte** (SVM), que el material de la materia estudia junto a los árboles, buscan el hiperplano que separa las clases con el **margen máximo**; solo los puntos sobre el margen —los vectores de soporte— determinan la frontera. Con un **kernel** (lineal o de base radial, RBF) pueden trazar fronteras no lineales; en regresión (SVR) ajustan una función que tolere errores dentro de un tubo de ancho ϵ .

3.3.3. Redes neuronales artificiales

Una neurona artificial calcula una suma ponderada de sus entradas más un sesgo, $z = \sum w_i x_i + b$, y le aplica una **función de activación** $a = g(z)$. El **perceptrón** de una sola capa solo separa clases linealmente separables: no puede resolver, por ejemplo, el problema XOR. El **perceptrón multicapa** (MLP) agrega una o más **capas ocultas** entre la entrada y la salida, con activaciones

no lineales; con suficientes neuronas puede aproximar cualquier función continua (teorema de aproximación universal).

En la **propagación hacia adelante** (*feedforward*) los datos atraviesan la red capa por capa, desde la entrada hasta la salida, sin ciclos:

$$z^{(l)} = a^{(l-1)} \cdot W^{(l)} + b^{(l)} \quad a^{(l)} = g(z^{(l)})$$

Las funciones de activación usadas son **ReLU**, $g(z) = \max(0, z)$, en las capas ocultas —simple, no satura para valores positivos y evita el desvanecimiento del gradiente—; **lineal** en la salida de regresión, para no acotar el valor predicho; y **softmax**, $p_i = e^{z_i} / \sum_j e^{z_j}$, en la salida de clasificación, que convierte las salidas en probabilidades que suman 1.

El error de la red se mide con una **función de pérdida**: el error cuadrático medio (dividido entre dos) en regresión y la **entropía cruzada** en clasificación. A ambas se suma la penalización L2 de los pesos:

$$L = (1/2n) \sum (\hat{y} - y)^2 + (\lambda/2) \sum \|W\|^2 \quad L = -(1/n) \sum \sum_k y_k \log p_k + (\lambda/2) \sum \|W\|^2$$

El aprendizaje consiste en ajustar los pesos para reducir L . La **retropropagación del error** (*backpropagation*) calcula eficientemente el gradiente $\partial L / \partial W$ de todos los pesos aplicando la regla de la cadena desde la salida hacia la entrada. Con salida lineal y error cuadrático, o con softmax y entropía cruzada, el error de la capa de salida toma la misma forma, $\delta = (\hat{y} - y)/n$; desde ahí el error viaja hacia atrás:

$$\partial L / \partial W^{(l)} = a^{(l-1)T} \cdot \delta^{(l)} + \lambda W^{(l)} \quad \delta^{(l-1)} = (\delta^{(l)} \cdot W^{(l)T}) \odot g'(z^{(l-1)})$$

Con el gradiente se actualizan los pesos por **descenso de gradiente**. En la variante por **mini-lotes con momento**, cada actualización usa un subconjunto pequeño de ejemplos y acumula una «velocidad» que suaviza la trayectoria y acelera la convergencia, con tasa de aprendizaje η y momento μ :

$$v \leftarrow \mu \cdot v - \eta \cdot \partial L / \partial W \quad W \leftarrow W + v$$

Dos técnicas controlan el sobreajuste: la **regularización L2**, ya incluida en la pérdida, y la **parada temprana** (*early stopping*): se mide la pérdida sobre el conjunto de validación después de cada época y el entrenamiento se detiene cuando deja de mejorar durante un número de épocas (la *paciencia*), conservando los pesos de la mejor época. Los pesos iniciales se toman al azar con la **inicialización de He**, de varianza $2/n$ con n el número de entradas de la capa, adecuada para ReLU.

3.4. Métricas de evaluación

En **regresión**, con y_i el valor real, $\hat{y}_i$ el predicho y $\bar{y}$ la media:

- $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$: fracción de la variación del objetivo que explica el modelo (1 es perfecto; 0 equivale a predecir siempre la media).
- $MSE = (1/n) \Sigma(y_i - \hat{y}_i)^2$: error cuadrático medio; penaliza fuertemente los errores grandes.
- $RMSE = \sqrt{MSE}$: el mismo error en las unidades del objetivo.
- $MAE = (1/n) \Sigma|y_i - \hat{y}_i|$: error absoluto medio, menos sensible a errores extremos.
- $MedAE = \text{mediana}(|y_i - \hat{y}_i|)$: la mitad de los errores es menor que este valor; robusto a atípicos.

En **clasificación**, a partir de la **matriz de confusión** (conteo de cada combinación de clase real y predicha):

- **Exactitud**: proporción de aciertos sobre el total.
- **Precisión**: de los ejemplos predichos como una clase, qué proporción lo es realmente.
- **Recall** (sensibilidad): de los ejemplos que realmente son de una clase, qué proporción se detecta.
- $F1 = 2 \cdot \text{precisión} \cdot \text{recall} / (\text{precisión} + \text{recall})$: equilibrio entre ambas. El **promedio macro** da el mismo peso a cada clase —revela si se ignoran las minoritarias—; el **ponderado** pesa cada clase por su tamaño.

Toda métrica se interpreta frente a un **baseline**: el desempeño de una regla trivial sin aprendizaje. En regresión, predecir siempre la media del entrenamiento; en clasificación, responder siempre la clase mayoritaria. Un modelo que no supera su baseline no aprendió nada útil, por buena que parezca su cifra aislada.

3.5. Justificación de la elección de modelos

- **Dos variantes por familia**. Comparar una variante simple y una con más capacidad dentro de cada familia (lineal contra Ridge, árbol contra bosque, kernel lineal contra RBF, una capa contra dos) separa el efecto de la familia del efecto de la complejidad.
- **Perceptrón multicapa como modelo de redes neuronales**. Los dos conjuntos son **tabulares**: filas independientes con columnas numéricas y categóricas codificadas. Las

redes convolucionales (pensadas para imágenes) y las recurrentes (para secuencias) no aportan nada a este tipo de dato; el MLP es la arquitectura adecuada.

- **Implementación propia.** La red se programó desde cero con NumPy para que el feedforward, la retropropagación y el descenso de gradiente sean código visible y verificable, no una llamada a una librería. Su gradiente se comprobó numéricamente (sección 4.6.2).
- **Modelos de scikit-learn como referencia.** Los ocho modelos de la librería validan la implementación propia: si la red desde cero obtiene métricas equivalentes al MLP de scikit-learn sobre los mismos datos, la implementación es correcta. Así ocurre: R^2 de prueba 0,9632 contra 0,9626.
- **Baseline obligatorio.** Cada resultado se reporta junto a su baseline, porque en Customer.csv una exactitud de 0,49 parece aceptable y en realidad es peor que no entrenar nada.

IV. Ingeniería

4.1. Descripción, ampliación y abstracción del problema

El análisis de datos de clientes es parte de la inteligencia de negocios: las organizaciones buscan segmentar a sus clientes para dirigir mejor sus estrategias comerciales. El análisis epidemiológico, por su parte, necesita estimar la magnitud de un brote a partir de los datos que efectivamente se reportan. En ambos casos, un modelo solo es útil si las variables disponibles contienen información sobre lo que se quiere predecir; parte del trabajo de ingeniería es verificar si esa relación existe.

4.1.1. Problema 1: fallecidos por COVID-19 (regresión)

Descripción. covid_19_data.csv contiene 306.429 registros diarios acumulados, por país y en algunos casos por provincia, desde el 22 de enero de 2020 hasta el 29 de mayo de 2021, con los casos confirmados, recuperados y fallecidos. **Ampliación.** Se busca estimar cuántos fallecidos acumulados corresponden a un registro, dados su país, su fecha y sus casos confirmados y recuperados; es útil, por ejemplo, para detectar registros cuyo número de fallecidos se aparta de lo esperable. **Abstracción.** Es una regresión:

$$f : (\text{país, día, } \log(1 + \text{confirmados}), \log(1 + \text{recuperados})) \rightarrow \log(1 + \text{fallecidos})$$

El modelo estima los fallecidos del mismo registro; no pronostica fallecidos futuros, porque confirmados y recuperados se conocen en la misma fecha que los fallecidos.

4.1.2. Problema 2: segmento comercial de un cliente (clasificación)

Descripción. Customer.csv contiene 793 clientes con su identificador, nombre, segmento, edad, país, ciudad, estado, código postal y región. **Ampliación.** Se busca asignar un cliente nuevo a uno de tres segmentos —*Consumer*, *Corporate* o *Home Office*— a partir de su edad y ubicación, y, antes que eso, comprobar si esas variables contienen información sobre el segmento. **Abstracción.** Es una clasificación multiclase:

$$f : (\text{edad, ciudad, estado, código postal, región}) \rightarrow \text{segmento} \in \{\text{Consumer, Corporate, Home Office}\}$$

Tabla 1. Características de los dos problemas

Aspecto	covid_19_data.csv	Customer.csv
Registros originales	306.429 (muestra de 20.000)	793
Columnas originales	8	9
Tarea	Regresión	Clasificación multiclase (3 clases)
Variable objetivo	Deaths, en escala $\log(1 + x)$	Segment
Variables predictoras	País, día de observación, confirmados, recuperados	Edad, ciudad, estado, código postal, región
Columnas tras la conversión	197	608
Baseline	RMSE 2,8052 (predecir la media)	Exactitud 0,5097 (responder «Consumer»)

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.2. Tecnologías utilizadas

Tabla 2. Tecnologías del proyecto y justificación de su uso

Tecnología	Versión	Uso y justificación
Python	3.12	Lenguaje de todo el análisis. Se fijó la versión 3.12 porque scikit-learn aún no publica paquetes estables para Python 3.14.
pandas	3.0.5	Lectura, limpieza y codificación de los datos (dropna, drop_duplicates, get_dummies). Todo el pre-procesamiento es código propio, sin herramientas externas.
NumPy	2.5.3	Álgebra matricial de la red neuronal desde cero: feedforward, backpropagation y actualización de pesos.
scikit-learn	1.9.1	Los ocho modelos de referencia, la partición de los datos y el cálculo de métricas.
uv	—	Crea el entorno virtual .venv con versiones fijas, para que cualquier integrante reproduzca los mismos resultados.
HTML, CSS y JavaScript	—	Dashboard de resultados, página de predicción y aplicación. Sin frameworks: la red entrenada se exporta a la página y el navegador repite el feedforward con los mismos pesos.
http.server (Python)	estándar	Servidor local de la aplicación: ejecuta los scripts de entrenamiento y transmite su salida en vivo.
Node.js / npm	—	Solo como lanzador: npm run dev inicia el servidor de Python y abre el navegador.
Git	—	Control de versiones del código y de los resultados de referencia.

Fuente: Elaboración propia a partir de los resultados del proyecto.

A diferencia de los avances anteriores, la versión final se ejecuta en un entorno local con versiones fijadas y semilla aleatoria 42 en el muestreo, la partición y cada modelo: dos ejecuciones producen exactamente las mismas cifras.

4.3. Arquitectura del sistema

El sistema es una cadena de scripts de Python que leen los datos, los preparan, entrenan los modelos y escriben sus resultados en la carpeta resultados/; las páginas web se generan a partir de esos archivos, de modo que ninguna cifra está escrita a mano. La Figura 1 resume el flujo.

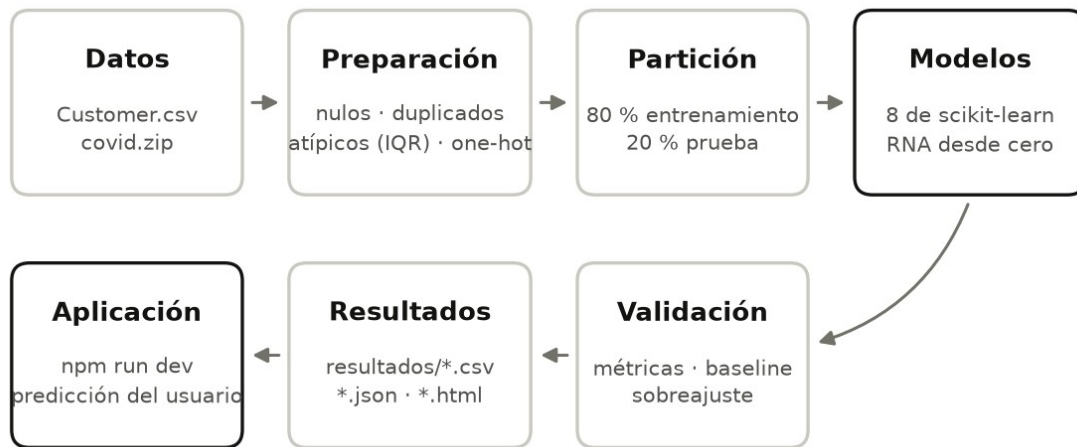


Figura 1. Arquitectura del sistema: de los datos crudos a la aplicación de predicción.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Tabla 3. Scripts del proyecto

Script	Función	Salida en resultados/
aprendizaje_supervisado.py	Pre-procesamiento de ambos datasets y los ocho modelos	resultados_*.csv, resumen.json, limpieza.json
diagnostico_customer.py	Cuatro experimentos sobre la señal de Customer.csv	diagnostico_customer.txt
enfoques_customer.py	Ocho formas alternativas de preparar Customer.csv	enfoques_customer.csv
funciones.py	Ajusta las funciones de los apuntes sobre datos reales	funciones.json
red_neuronal.py	Red neuronal desde cero: entrenamiento, validación y predicción	red_neuronal.json, red_neuronal.html
test_red_neuronal.py	Verificación del gradiente y del aprendizaje de XOR	—
dashboard.py	Genera el dashboard de los ocho modelos	dashboard.html
servidor.py + app.html	Aplicación: entrenar y ver resultados en una ventana	—

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.4. Pre-procesamiento de datos

Los tres procesos se aplican a ambos conjuntos con una misma función, `limpiar()`, que antes de cada corrección imprime la «tabla de fallos» con las filas detectadas y registra cada etapa en `resultados/limpieza.json`. Todo el pre-procesamiento está escrito en el propio código con `pandas`; no se usó ninguna herramienta externa de limpieza.

4.4.1. Proceso 1: incompletitud y técnica aplicada a los datos faltantes

Tabla 4. Valores nulos por columna en `covid_19_data.csv` (archivo completo)

Columna	Nulos	% de filas
SNo	0	0,0 %
ObservationDate	0	0,0 %
Province/State	78.103	25,5 %
Country/Region	0	0,0 %
Last Update	0	0,0 %
Confirmed	0	0,0 %
Deaths	0	0,0 %
Recovered	0	0,0 %

Fuente: Elaboración propia a partir de los resultados del proyecto.

En `Customer.csv` no hay ningún valor nulo. En `covid_19_data.csv` la única columna incompleta es `Province/State`, vacía en 78.103 registros (25,5 %).

Técnica aplicada: eliminación de la columna, no imputación. La ausencia de provincia no es un dato perdido: indica que el país reporta sus cifras a nivel nacional. Es una ausencia **estructural**, y por eso ninguna técnica de imputación es correcta: imputar la moda asignaría a todos esos países la provincia más frecuente de otro país, e imputar una constante («Sin provincia») crearía una categoría artificial. Tampoco se eliminaron las filas: `dropna()` con la columna presente habría borrado la cuarta parte del conjunto, justamente los países que reportan a nivel nacional. Como el país ya describe la ubicación, se eliminó la columna y se conservaron todas las filas. Después de eso, ninguna otra columna tiene nulos, por lo que el `dropna()` general del código —que se mantiene por robustez ante datos nuevos— no elimina ninguna fila.

4.4.2. Proceso 2: ruido y excepciones

Duplicados. Se detectan con `duplicated(keep=False)`, que marca todas las copias de cada fila repetida, y se eliminan con `drop_duplicates()`, que conserva una. En `Customer.csv`, 33 filas

participan en duplicados y se eliminan 18; en la muestra de covid, 36 y 21. Un duplicado exacto no aporta información y sobrepondera ese ejemplo en el entrenamiento; además, si una copia cae en entrenamiento y otra en prueba, la evaluación se contamina.

Valores imposibles. Un conteo acumulado no puede ser negativo; el archivo completo contiene valores negativos de Confirmed (el mínimo es -302.844 , un error de captura de la fuente), que el código descarta. En la muestra usada no cayó ninguno.

Atípicos: transformación $\log(1 + x)$ antes del IQR. Los conteos tienen una cola muy larga: en la muestra, la media de Confirmed es 85.548, la mediana 10.529 y el máximo 5.698.706, con una asimetría de 8,46. Aplicado directamente, el IQR marcaría como atípicos a todos los registros de los países grandes, que son legítimos. Por eso se aplica primero $\log(1 + x)$ a Confirmed, Deaths y Recovered: la asimetría baja a -0,74 (Figura 2) y el IQR solo marca lo verdaderamente anómalo. Se usa $\log(1 + x)$ y no $\log(x)$ porque hay conteos iguales a cero.

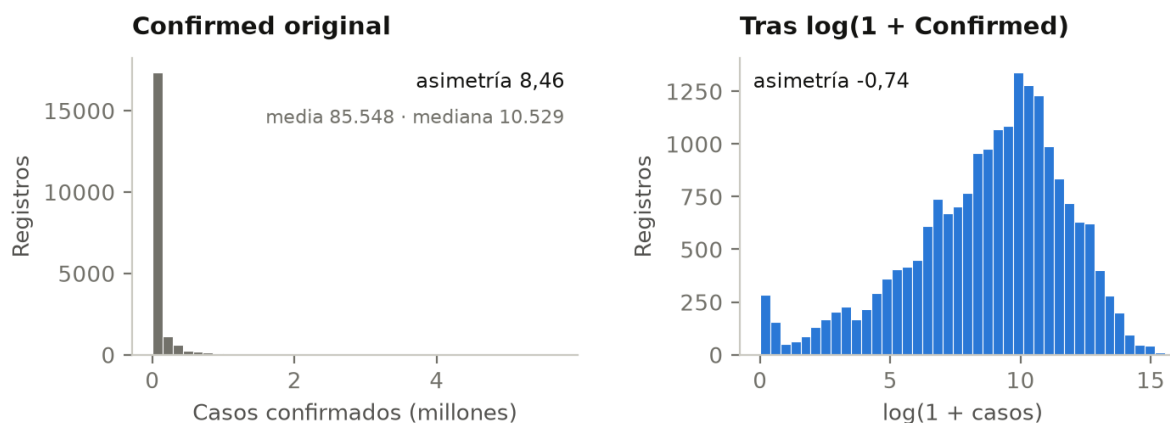


Figura 2. Distribución de los casos confirmados antes y después de la transformación $\log(1 + x)$.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Con la escala logarítmica, el filtro IQR elimina 476 registros de covid; en Customer.csv, la única variable numérica, la edad (entre 18 y 70 años), no presenta atípicos. La transformación se mantiene después del filtro porque además es la adecuada para modelar conteos: el modelo aprende relaciones proporcionales («el doble de casos») en lugar de absolutas.

4.4.3. Proceso 3: conversión de datos

Tabla 5. Cardinalidad de las columnas de Customer.csv y decisión tomada

Columna	Valores distintos	Decisión
Customer ID	793	Se elimina: un valor distinto por fila; no generaliza a clientes nuevos y generaría 793 columnas inútiles.
Customer Name	793	Se elimina por el mismo motivo.
Country	1	Se elimina: constante (todos son de Estados Unidos), no aporta información.
Segment	3	Variable objetivo; se separa antes de codificar.
Age	53	Númerica; se conserva.
City	252	One-hot.
State	41	One-hot.
Postal Code	314	Se trata como texto y se codifica con one-hot: es un código, no una cantidad (el 90036 no es «mayor» que el 42420).
Region	4	One-hot.

Fuente: Elaboración propia a partir de los resultados del proyecto.

En covid se eliminan SNo (un contador de filas) y Last Update (repite la información de la fecha de observación). La fecha ObservationDate se convierte en **día de la pandemia**: número de días desde el 22 de enero de 2020, una variable numérica que conserva el orden temporal. La única variable de texto restante, Country/Region, se codifica con one-hot.

La codificación usa `pd.get_dummies(drop_first=True)`: la primera categoría de cada variable queda representada por todas sus columnas en cero, lo que evita la redundancia exacta entre columnas (una columna sería combinación lineal de las demás). Customer.csv pasa de 5 columnas predictoras a **608** (la edad más 607 columnas binarias) y covid a **197** (confirmados, recuperados y día, más 194 países). La primera fila de Customer.csv, por ejemplo —Henderson, Kentucky, 42420, South—, queda como un 1 en las columnas City_Henderson, State_Kentucky, Postal Code_42420 y Region_South, y 0 en las otras 603 columnas binarias.

4.4.4. Estandarización

Todas las variables se estandarizan a media 0 y desviación 1. La media y la desviación se calculan **solo con el conjunto de entrenamiento** y se aplican después a validación, prueba y a los datos que ingrese el usuario: usarlas sobre todo el conjunto filtraría información de prueba al entrenamiento. En los modelos de scikit-learn el escalado va dentro de un Pipeline con

StandardScaler; en la red propia lo hace la clase Estandarizador. A las columnas constantes se les asigna desviación 1 para no dividir entre cero.

4.4.5. Correcciones al código del primer avance

Al integrar el pre-procesamiento del primer avance con los modelos se corrigieron cuatro puntos del código original:

- **Selección del objetivo.** El código tomaba como objetivo la última columna *después* de aplicar `get_dummies()`, que resultaba ser una variable dummy arbitraria (`Region_West`). Ahora el objetivo se nombra explícitamente y se separa antes de codificar.
- **Identificadores.** Customer ID y Customer Name se codificaban con one-hot; ahora se eliminan antes (sección 4.4.3).
- **Nulos.** `dropna()` sobre todas las columnas borraba un cuarto de covid; ahora se elimina la columna Province/State (sección 4.4.1).
- **Atípicos.** El IQR sobre los conteos crudos marcaba como atípico a casi todo el dataset; ahora se aplica sobre $\log(1 + x)$ (sección 4.4.2).

La Figura 3 resume cuántas filas conserva cada etapa. El pre-procesamiento deja 775 clientes y 19.503 registros de covid.

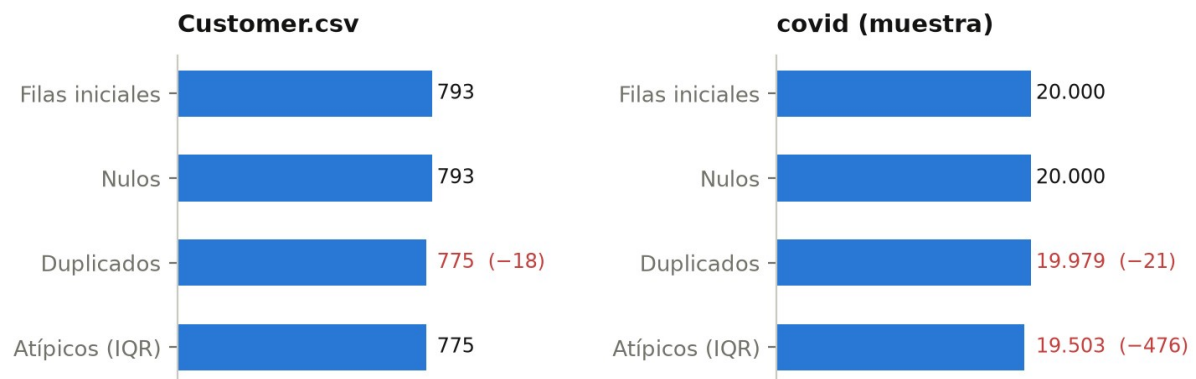


Figura 3. Filas que conserva cada etapa del pre-procesamiento; entre paréntesis, las eliminadas.

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.5. Determinación de los datos de entrenamiento

4.5.1. Variables predictoras y objetivo

Tabla 6. Variables de covid_19_data.csv y justificación de su rol

Variable	Rol	Justificación
Deaths	Objetivo	Continua y de interés directo. Se modela $\log(1 + \text{Deaths})$ porque los valores van de 0 a más de medio millón: en escala original, los pocos registros enormes dominarían el error.
Confirmed	Predictora	Relación epidemiológica directa: más casos implican más fallecidos. En escala $\log(1 + x)$.
Recovered	Predictora	Indica la evolución del brote en ese registro y la capacidad de reporte. En escala $\log(1 + x)$.
Día de la pandemia	Predictora	La letalidad cambió con el tiempo (tratamientos, vacunación, saturación hospitalaria); el día permite capturar esa etapa.
Country/Region	Predictora	El sistema de salud, la estructura de edad y la forma de reportar difieren por país. 194 columnas binarias.
Province/State	Excluida	Ausencia estructural en el 25,5 % de las filas (sección 4.4.1).
SNo, Last Update	Excluidas	Contador sin significado y fecha redundante.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Tabla 7. Variables de Customer.csv y justificación de su rol

Variable	Rol	Justificación
Segment	Objetivo	Es la variable de negocio de interés. Region fue la otra candidata, pero queda determinada por el estado (California siempre es West): predecirla es trivial y no aporta.
Age	Predictora	Única variable demográfica disponible; hipótesis: la edad podría relacionarse con el tipo de cliente.
City, State, Postal Code, Region	Predictoras	La ubicación podría relacionarse con el segmento (zonas corporativas o residenciales).
Customer ID, Customer Name, Country	Excluidas	Identificadores únicos y una constante (sección 4.4.3).

Fuente: Elaboración propia a partir de los resultados del proyecto.

Muestra de covid. Se trabaja con una muestra aleatoria de 20.000 registros (semilla 42) en lugar de los 306.429. La razón es de costo computacional: el costo de entrenar un SVM crece de forma aproximadamente cuadrática con el número de muestras y las redes neuronales recorren los datos cientos de veces; sobre el conjunto completo el SVR no termina en un tiempo razonable. Al ser aleatoria, la muestra no privilegia ningún país ni período: tras la limpieza cubre 195 países y toda la ventana de fechas del archivo.

4.5.2. División en entrenamiento, validación y prueba

Tabla 8. *Partición de los datos*

Conjunto	Proporción	covid	Customer	Uso
Entrenamiento (8 modelos)	80 %	15.602	620	Ajuste de los modelos de scikit-learn
Prueba (todos)	20 %	3.901	155	Solo para medir las métricas finales
Entrenamiento (RNA)	68 %	13.261	527	Ajuste de los pesos de la red
Validación (RNA)	12 %	2.341	93	Early stopping y selección de hiperparámetros

Fuente: Elaboración propia a partir de los resultados del proyecto.

¿Por qué 80/20? Es la proporción estándar: deja la mayor parte de los datos para aprender y un conjunto de prueba suficiente para medir con estabilidad (3.901 registros en covid). En Customer.csv el conjunto es pequeño; un 20 % de prueba (155 clientes) es el mínimo razonable para que la exactitud no dependa de unos pocos casos, y por eso no se usó una proporción menor.

Estratificación. En Customer.csv la partición es estratificada por segmento: entrenamiento y prueba conservan la misma proporción de Consumer, Corporate y Home Office (51,6 %, 29,8 % y 18,7 %). Sin estratificar, con solo 155 clientes de prueba, el azar podría dejar la clase minoritaria sub-representada y sesgar la exactitud.

Validación para la red neuronal. La red necesita tomar decisiones durante el entrenamiento —cuándo detenerse y qué hiperparámetros usar—. Si esas decisiones se tomaran mirando el conjunto de prueba, las métricas finales estarían sesgadas a favor del modelo. Por eso el 80 % de entrenamiento se divide otra vez en 85 % de entrenamiento y 15 % de validación (68 % y 12 % del total). El 20 % de prueba es el mismo para todos los modelos, lo que hace comparables sus métricas.

4.6. Modelos implementados

4.6.1. Ocho modelos de referencia

Tabla 9. Modelos de referencia, hiperparámetros y justificación

Familia	Modelo	Hiperparámetros	Justificación
Regresión	Lineal / Logística (softmax)	por defecto	Línea base lineal: indica cuánto se explica sin interacciones ni no linealidad.
Regresión	Ridge (L2)	$\alpha = 1,0$	Mide si el modelo lineal sobreajusta: si Ridge no mejora, el problema es de capacidad y no de varianza.
Decisión	Árbol de decisión	profundidad 8 (regresión) / 5 (clasificación)	Captura no linealidad e interacciones con reglas interpretables; la profundidad limita la memorización.
Decisión	Random Forest	200 árboles	Reduce la varianza del árbol individual promediando; 200 árboles estabilizan el resultado sin costo excesivo.
Vectores	SVM / SVR kernel lineal	$C = 1$	Frontera de margen máximo lineal.
Vectores	SVM / SVR kernel RBF	$C = 1, \gamma = \text{scale}$	Frontera no lineal mediante el kernel.
Red neuronal	MLP de scikit-learn, 1 capa	32 neuronas, ReLU, Adam	Referencia para validar la red propia.
Red neuronal	MLP de scikit-learn, 2 capas	64 y 32 neuronas	Mide el efecto de agregar profundidad.

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.6.2. Red neuronal artificial desde cero

La red se implementó en la clase `PerceptronMulticapa` de `src/red_neuronal.py`, usando solo NumPy. Se entrenan dos redes, una por problema. La Figura 4 muestra la arquitectura de la red de covid.

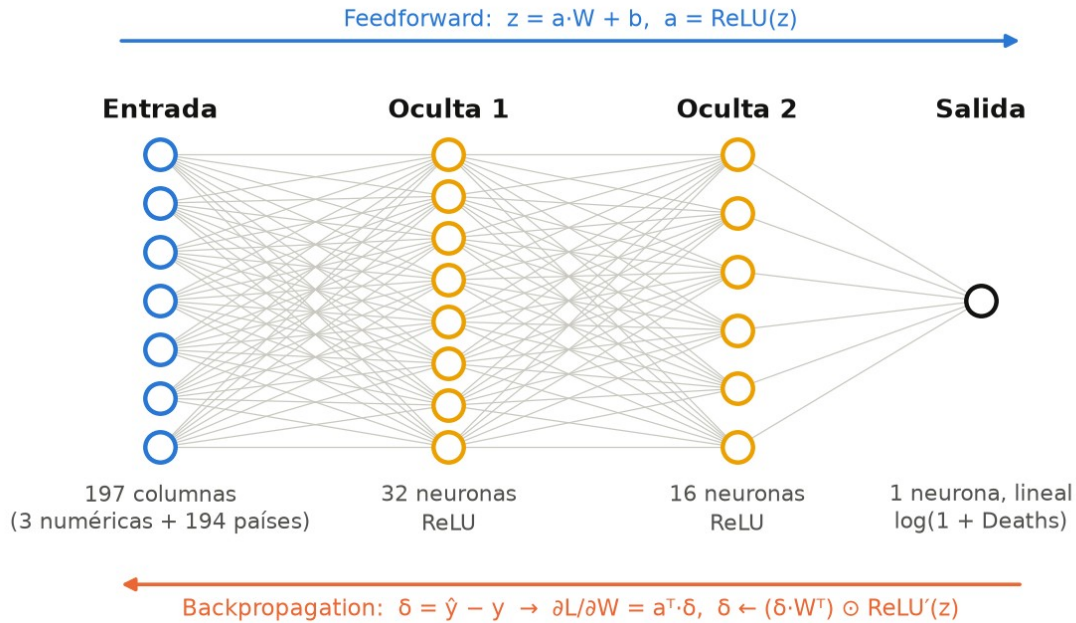


Figura 4. Arquitectura del perceptrón multicapa para covid (se dibuja un subconjunto de las neuronas de cada capa).

Fuente: Elaboración propia a partir de los resultados del proyecto.

Tabla 10. Arquitectura de las dos redes

Capa	Red covid (regresión)	Red Customer (clasificación)
Entrada	197 neuronas (variables estandarizadas)	608 neuronas
Oculto 1	32 neuronas, ReLU	16 neuronas, ReLU
Oculto 2	16 neuronas, ReLU	—
Salida	1 neurona, activación lineal: $\log(1 + \text{Deaths})$	3 neuronas, softmax: probabilidad de cada segmento
Pérdida	Error cuadrático medio / 2 + L2	Entropía cruzada + L2
Parámetros entrenables	6.881	9.795

Fuente: Elaboración propia a partir de los resultados del proyecto.

Algoritmo de entrenamiento. Para cada red:

1. Inicializar los pesos al azar con la inicialización de He y los sesgos en cero.
2. En cada época, barajar las filas de entrenamiento y recorrerlas en mini-lotes (64 en covid, 32 en Customer).
3. Para cada mini-lote: propagar hacia adelante (feedforward), calcular el error de salida, retropropagarlo capa por capa para obtener los gradientes y actualizar los pesos con momento.

4. Al terminar la época, calcular la pérdida sobre entrenamiento y validación. Si la pérdida no es finita, detener con un error explícito (la tasa de aprendizaje es demasiado alta).
5. Si la pérdida de validación es la mejor hasta el momento, guardar una copia de los pesos. Si no mejora durante 25 épocas seguidas (paciencia), detener el entrenamiento.
6. Restaurar los pesos de la mejor época de validación.

El núcleo del algoritmo son dos métodos. El feedforward guarda la activación de cada capa, porque la retropropagación las necesita; la retropropagación implementa las ecuaciones de la sección 3.3.3:

```
def propagar(self, X):          # feedforward
    activaciones = [X]
    for i, (W, b) in enumerate(zip(self.W, self.b)):
        z = activaciones[-1] @ W + b
        es_ultima = i == len(self.W) - 1
        activaciones.append(self._salida(z) if es_ultima else np.maximum(z, 0))
    return activaciones

def retropropagar(self, activaciones, Y): # backpropagation
    delta = (activaciones[-1] - Y) / len(Y)
    grad_W, grad_b = [None] * len(self.W), [None] * len(self.b)
    for i in reversed(range(len(self.W))):
        grad_W[i] = activaciones[i].T @ delta + self.alfa * self.W[i]
        grad_b[i] = delta.sum(axis=0)
        if i > 0:
            delta = (delta @ self.W[i].T) * (activaciones[i] > 0)
    return grad_W, grad_b

# actualizacion con momento, para cada capa i
vel_W[i] = self.momento * vel_W[i] - self.tasa * grad_W[i]
self.W[i] += vel_W[i]
```

Tabla 11. *Hiperparámetros de entrenamiento y justificación*

Hiperparámetro	covid	Customer	Justificación
Tasa de aprendizaje η	0,005	0,01	Covid actualiza los pesos unas 200 veces por época, por lo que usa un paso pequeño y converge en 108 épocas. En Customer, 0,01 y 0,001 llegan a la misma pérdida de validación, pero 0,01 en 26 épocas en lugar de 207.
Momento μ	0,9	0,9	Valor estándar: suaviza el ruido de los mini-lotes y acelera la convergencia.
Tamaño de mini-lote	64	32	Equilibrio entre la estabilidad del gradiente y la cantidad de actualizaciones; Customer tiene pocas filas.
Regularización L2 λ	0,0001	1,0	Seleccionada por la pérdida de validación (tablas siguientes). Customer necesita una penalización fuerte: 527 filas frente a 608 columnas.
Paciencia (early stopping)	25	25	Tolera las oscilaciones normales de la pérdida sin entrenar indefinidamente.
Épocas máximas	500	500	Límite de seguridad; el early stopping detiene antes (133 y 51 épocas).

Fuente: Elaboración propia a partir de los resultados del proyecto.

Selección de hiperparámetros. Se compararon varias configuraciones usando **solo la pérdida de validación**; el conjunto de prueba no intervino en ninguna decisión. En covid se probaron cuatro arquitecturas:

Tabla 12. *Selección de la arquitectura de la red de covid (tasa 0,005)*

Capas ocultas	Mejor época	Pérdida de validación	R ² de validación
16 neuronas	154	0,1255	0,9686
32 neuronas	103	0,1293	0,9678
32 y 16 neuronas	108	0,1151	0,9721
64 y 32 neuronas	128	0,1207	0,9717

Fuente: Elaboración propia a partir de los resultados del proyecto.

La arquitectura de 32 y 16 neuronas obtiene la menor pérdida de validación y es la elegida. Una red más grande (64 y 32) no mejora: tiene cuatro veces más parámetros y valida peor, señal de que la capacidad adicional ya no aporta.

Tabla 13. Selección de la regularización de la red de Customer (16 neuronas ocultas)

Tasa η	L2 λ	Mejor época	Pérdida de validación	Exactitud de validación
0,010	0,01	1	1,5153	0,4731
0,010	0,10	15	1,3682	0,4731
0,010	1,00	26	1,0134	0,5161
0,001	0,01	6	1,4326	0,3871
0,001	0,10	140	1,3858	0,4516
0,001	1,00	207	1,0135	0,5161

Fuente: Elaboración propia a partir de los resultados del proyecto.

Con L2 de 0,01 o 0,1 la pérdida de validación empieza a subir desde las primeras épocas —la red memoriza clientes—; con 1,0 alcanza su mínimo (1,0134), que coincide con la entropía de la distribución de segmentos: la red aprende a responder la proporción de cada clase. Ese resultado se analiza en la sección 4.7.5.

Verificación de la implementación. Una implementación de backpropagation puede tener errores que no impiden entrenar pero sí degradan el aprendizaje. `src/test_red_neuronal.py` compara el gradiente analítico con el gradiente numérico por diferencias finitas centradas, $[L(w + \epsilon) - L(w - \epsilon)] / 2\epsilon$ con $\epsilon = 10^{-6}$, para cada peso de una red pequeña, en regresión y en clasificación; el error relativo es menor que 10^{-5} . El test falla si se altera el gradiente en un 1 %, lo que confirma que detecta errores reales. Además verifica que la red aprende el problema XOR, imposible para un perceptrón de una sola capa. Por último, la página web repite el feedforward en JavaScript con los pesos exportados, y sus predicciones coinciden con las de Python con una diferencia máxima de $2 \cdot 10^{-6}$.

4.7. Validación con cinco métricas

4.7.1. Métricas elegidas y su justificación

Tabla 14. Métricas de validación y por qué se usa cada una

Tarea	Métrica	Por qué se usa
Regresión	R^2	Resume en una cifra qué parte de la variación se explica; es comparable entre modelos y frente al baseline ($R^2 = 0$).
Regresión	MSE	Es la pérdida que minimiza la red; penaliza los errores grandes.
Regresión	RMSE	El error típico en la escala del objetivo; se compara directamente con el baseline de predecir la media.
Regresión	MAE	Error promedio menos sensible a los registros extremos que el RMSE.
Regresión	MedAE	Error del caso típico; en escala logarítmica, e^{MedAE} da el factor de error de la mitad de las predicciones.
Clasificación	Exactitud	La métrica que se compara con el baseline de la clase mayoritaria.
Clasificación	Precisión macro	Revela si el modelo acierta cuando predice cada clase, con el mismo peso para las minoritarias.
Clasificación	Recall macro	Revela si el modelo encuentra los ejemplos de cada clase. Un valor de 0,3333 con tres clases indica que solo predice una.
Clasificación	F1 macro	Combina las dos anteriores; es la métrica adecuada con clases desbalanceadas.
Clasificación	F1 ponderado	F1 pesado por el tamaño de cada clase: muestra el desempeño sobre la población real.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Todas las métricas se calculan sobre el mismo 20 % de prueba, con datos que ningún modelo vio al entrenar.

4.7.2. Resultados en covid_19_data.csv

Tabla 15. Regresión de $\log(1 + \text{Deaths})$: cinco métricas en prueba y tiempo de entrenamiento

Modelo	Familia	R^2	MSE	RMSE	MAE	MedAE	Seg.
Random Forest (200 árboles)	Árboles	0,9686	0,2473	0,4973	0,3075	0,1915	4,90
RNA desde cero (32, 16)	Red neuronal	0,9632	0,2896	0,5382	0,3326	0,2177	≈ 16
MLP 2 capas (64,32)	Red neuronal	0,9626	0,2940	0,5422	0,3631	0,2537	18,80
MLP 1 capa (32)	Red neuronal	0,9586	0,3259	0,5709	0,3772	0,2617	11,91
Árbol de decisión (prof. 8)	Árboles	0,9251	0,5890	0,7675	0,5290	0,3773	0,22
Regresión lineal	Regresión	0,9246	0,5931	0,7702	0,5156	0,3650	0,49
Regresión Ridge (L2)	Regresión	0,9246	0,5932	0,7702	0,5156	0,3650	0,25
SVR kernel lineal	Vectores de soporte	0,9212	0,6199	0,7874	0,4977	0,3269	75,92
SVR kernel RBF	Vectores de soporte	0,8642	1,0682	1,0335	0,6112	0,3510	58,35

Fuente: Elaboración propia a partir de los resultados del proyecto.

Baseline (predecir siempre la media): RMSE 2,8052. Los nueve modelos lo superan ampliamente. La Figura 5 compara el R^2 .

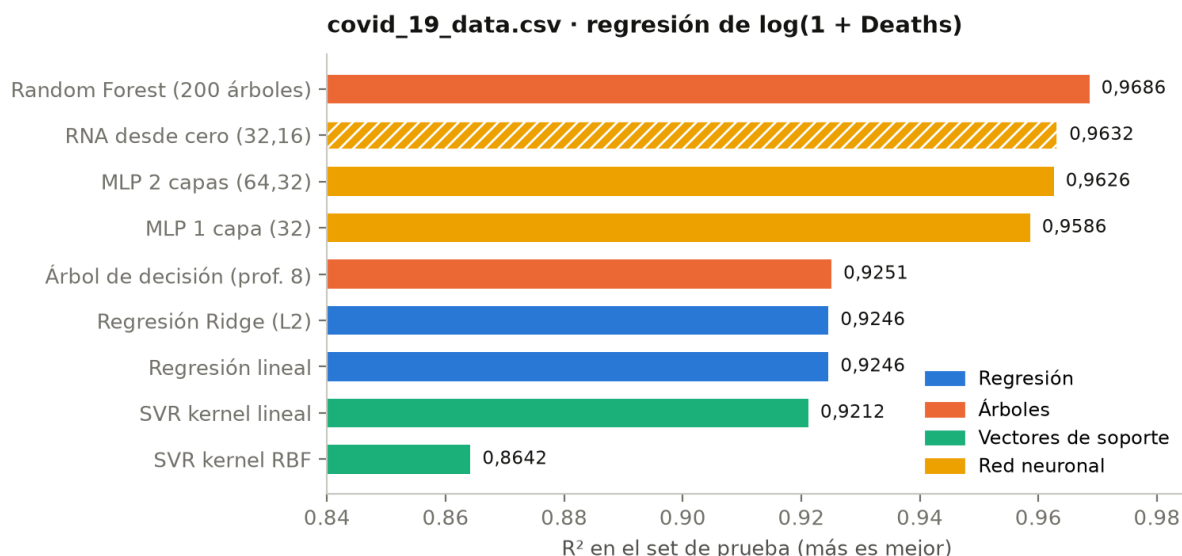


Figura 5. R^2 en el conjunto de prueba de covid, por modelo y familia. La barra rayada es la red neuronal propia.

Fuente: Elaboración propia a partir de los resultados del proyecto.

- **Los árboles ganan porque la relación no es lineal.** La regresión lineal y Ridge obtienen el mismo R^2 (0,9246) hasta el cuarto decimal: la regularización no cambia nada, lo que indica que el modelo lineal no sobreajusta sino que le falta capacidad. Random Forest (0,9686) captura la no linealidad y gana 4,4 puntos de R^2 .
- **Las redes neuronales quedan a la par.** La red desde cero (0,9632) y el MLP de dos capas de scikit-learn (0,9626) quedan a 0,006 o menos del bosque. Que la implementación propia iguale a la de la librería valida su corrección.
- **El SVM no compete en este problema.** El kernel RBF queda último (0,8642) y el lineal necesita 76 segundos para quedar por debajo de la regresión lineal, que tarda medio segundo (Figura 6).

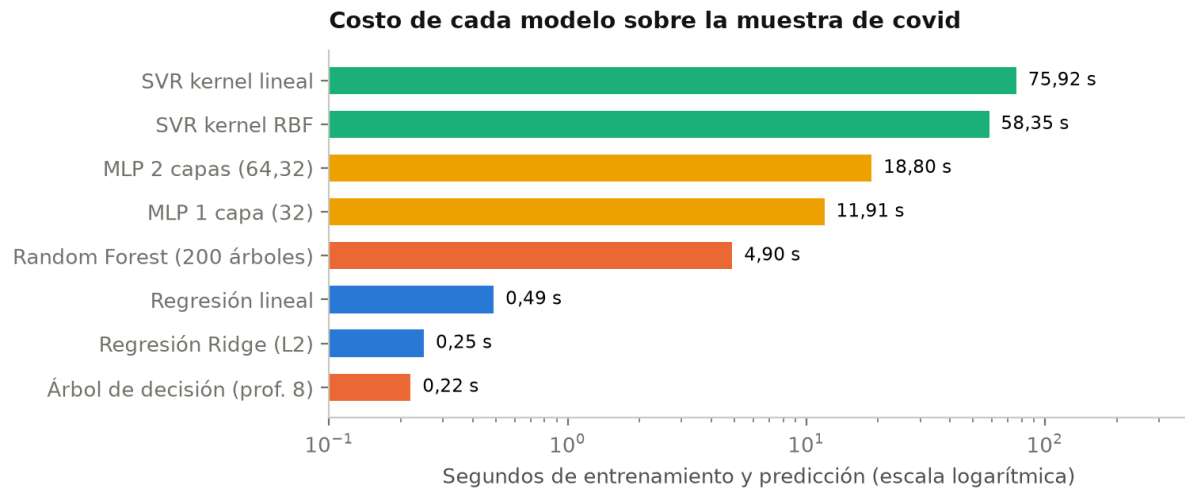


Figura 6. *Tiempo de entrenamiento y predicción de cada modelo sobre la muestra de covid (escala logarítmica).*

Fuente: Elaboración propia. Los tiempos dependen de la máquina; el orden entre modelos se mantiene.

La Figura 7 explica por qué ganan los árboles con las funciones de los apuntes de la materia, ajustadas sobre 3.000 registros reales de covid: la recta explica un R^2 de 0,823, una curva de grado 3 sube a 0,855 y los escalones de un árbol de solo 8 hojas llegan a 0,849. La relación entre casos y fallecidos se dobla, y los modelos capaces de seguir esa curva ganan.

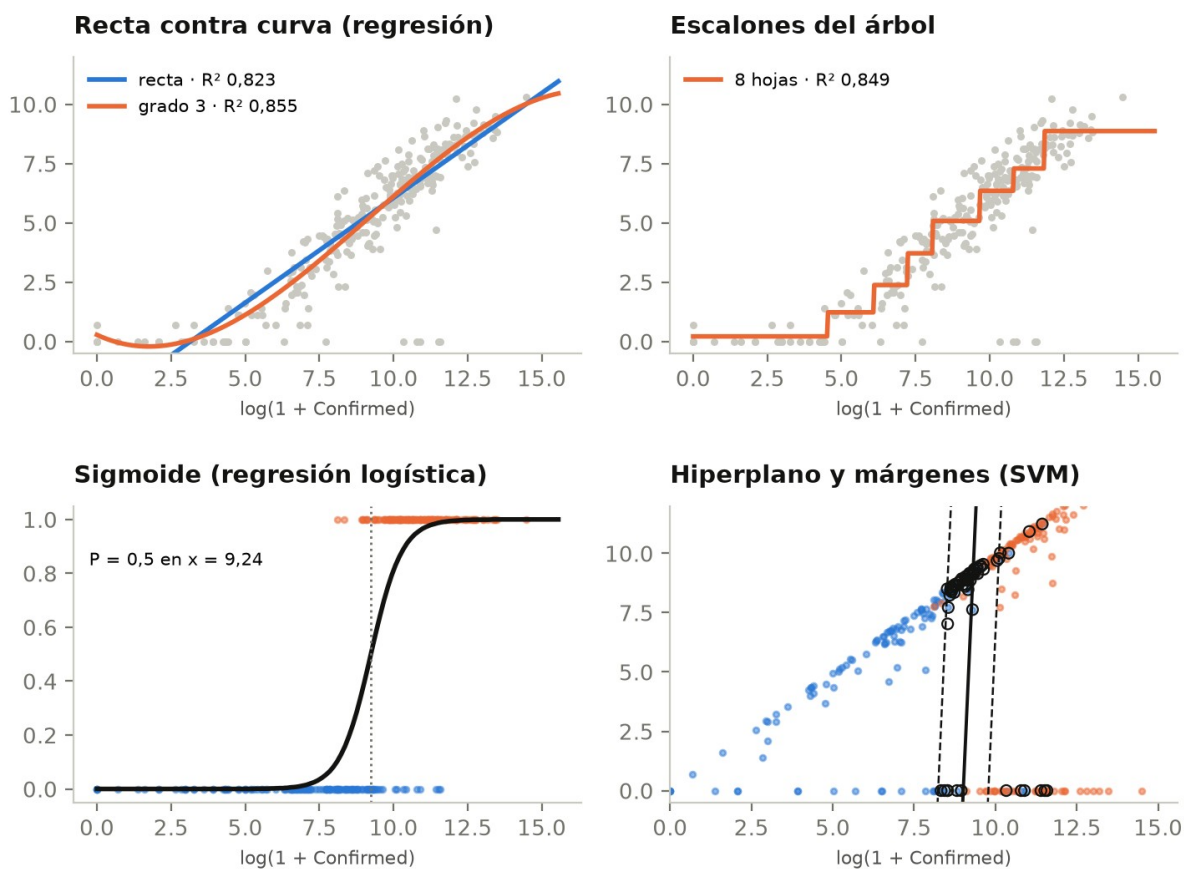


Figura 7. Funciones que ajusta cada familia sobre datos reales de covid: recta y curva, escalones del árbol, sigmoide de la regresión logística y margen del SVM.

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.7.3. Resultados en Customer.csv

Tabla 16. Clasificación de Segment: cinco métricas en prueba

Modelo	Familia	Exactitud	Precisión macro	Recall macro	F1 macro	F1 pond.
RNA desde cero (16)	Red neuronal	0,5097	0,1699	0,3333	0,2251	0,3441
Árbol de decisión (prof. 5)	Árboles	0,4903	0,1689	0,3207	0,2213	0,3383
SVM kernel lineal	Vectores de soporte	0,4710	0,3373	0,3440	0,3158	0,4097
SVM kernel RBF	Vectores de soporte	0,4581	0,2741	0,3139	0,2594	0,3633
Regresión logística (softmax)	Regresión	0,4323	0,3179	0,3231	0,3051	0,3876
Clasificador Ridge (L2)	Regresión	0,4323	0,3199	0,3231	0,3051	0,3869
MLP 1 capa (32)	Red neuronal	0,4194	0,3206	0,3262	0,3163	0,3918
Random Forest (200 árboles)	Árboles	0,4129	0,3009	0,3089	0,2963	0,3814
MLP 2 capas (64,32)	Red neuronal	0,3484	0,2792	0,2813	0,2789	0,3383

Fuente: Elaboración propia a partir de los resultados del proyecto.

Baseline (responder siempre «Consumer»): exactitud 0,5097. La Figura 8 lo marca con la línea roja.

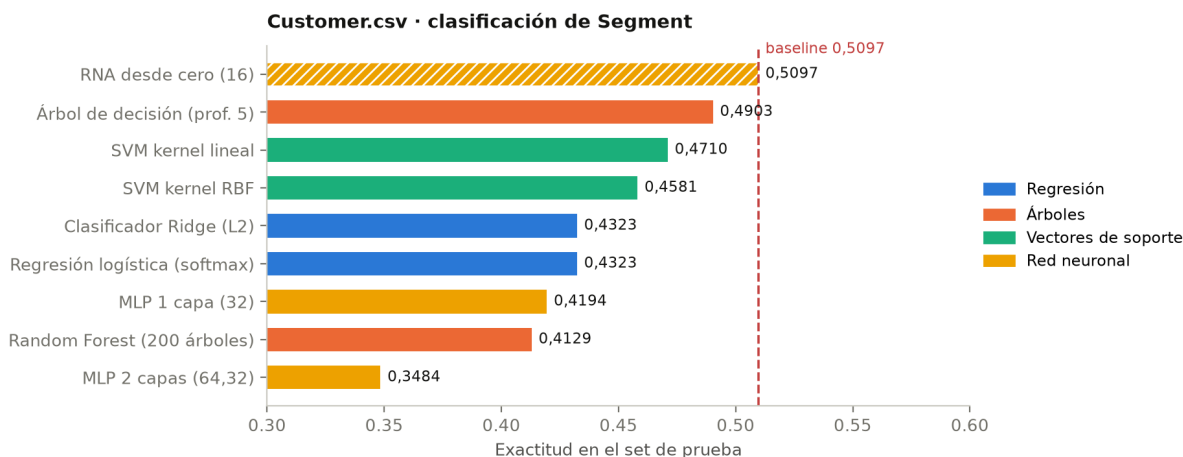


Figura 8. Exactitud en el conjunto de prueba de *Customer.csv* frente al baseline.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Ningún modelo de scikit-learn supera el baseline; el mejor, el árbol de profundidad 5, queda en 0,4903. La red desde cero lo **igual**a exactamente (0,5097), y la razón es reveladora: su recall macro es 0,3333 —uno de tres—, lo que significa que predice siempre la misma clase. Con la regularización que minimiza la pérdida de validación, la mejor estrategia que encuentra la red es responder la clase mayoritaria. Los modelos con más capacidad, como Random Forest y el MLP de dos capas, quedan por debajo del baseline porque aprenden ruido de 608 columnas con solo 620 filas.

4.7.4. Control del sobreajuste

La práctica pide el mejor ajuste sin sobreajuste. El indicador es la **brecha** entre el desempeño en entrenamiento y en prueba: un modelo que generaliza tiene métricas parecidas en ambos. La Figura 9 las compara para todos los modelos.

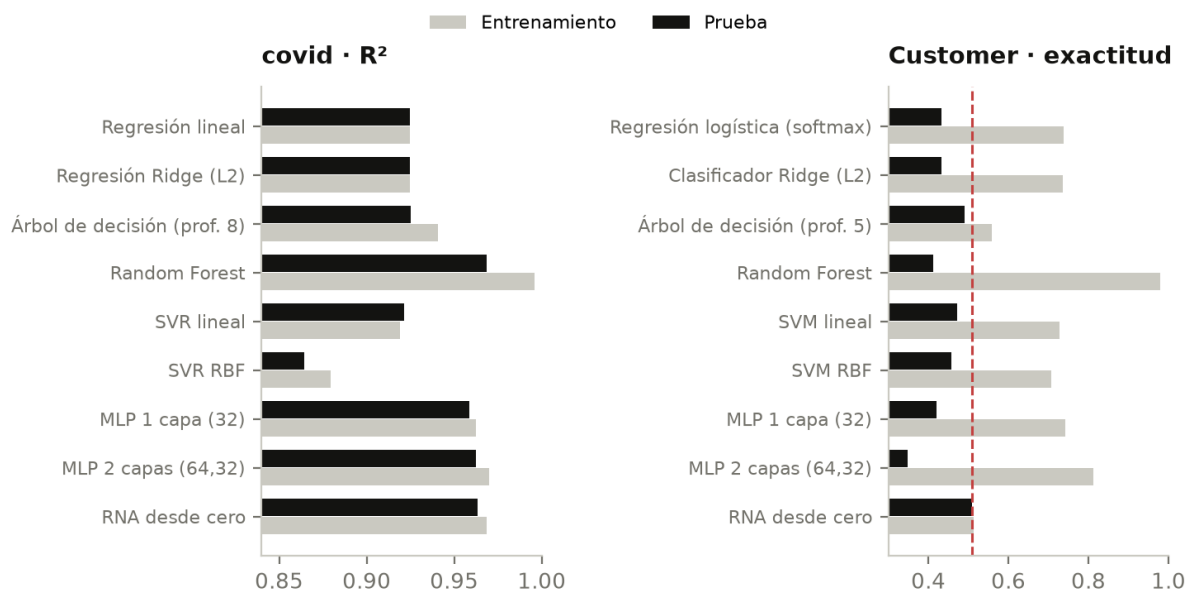


Figura 9. Desempeño en entrenamiento (gris) y en prueba (negro). La distancia entre ambas barras es el sobreajuste.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Tabla 17. Brecha entre entrenamiento y prueba en covid (R^2)

Modelo	R^2 entrenamiento	R^2 prueba	Brecha
Regresión lineal	0,9245	0,9246	-0,0001
Regresión Ridge (L2)	0,9245	0,9246	-0,0001
Árbol de decisión (prof. 8)	0,9407	0,9251	0,0156
Random Forest (200 árboles)	0,9959	0,9686	0,0273
SVR kernel lineal	0,9190	0,9212	-0,0022
SVR kernel RBF	0,8792	0,8642	0,0150
MLP 1 capa (32)	0,9623	0,9586	0,0037
MLP 2 capas (64,32)	0,9698	0,9626	0,0072
RNA desde cero	0,9683	0,9632	0,0051

Fuente: Elaboración propia a partir de los resultados del proyecto.

Random Forest obtiene el mejor R^2 de prueba, pero con una brecha de 0,0273: explica el 99,6 % del entrenamiento, señal de que memoriza parte de él. La red desde cero tiene una de las brechas más pequeñas, 0,0051, gracias al early stopping y a la regularización; entre los modelos que superan R^2 0,95 solo el MLP de una capa la tiene menor, a costa de un R^2 de prueba más bajo. En Customer.csv el sobreajuste es evidente en los modelos flexibles: Random Forest acierta el 97,9 % en entrenamiento y el 41,3 % en prueba.

Las curvas de pérdida de la red (Figura 10) muestran el early stopping en funcionamiento. En covid, la pérdida de validación sigue a la de entrenamiento —no se separan, que es la firma del sobreajuste— y alcanza su mínimo en la época 108; la franja gris son las 25 épocas de paciencia sin mejora, tras las cuales el entrenamiento se detiene y se restauran los pesos de la época 108. En Customer, ambas pérdidas se estabilizan en pocas épocas en el valor que corresponde a responder la proporción de cada segmento.

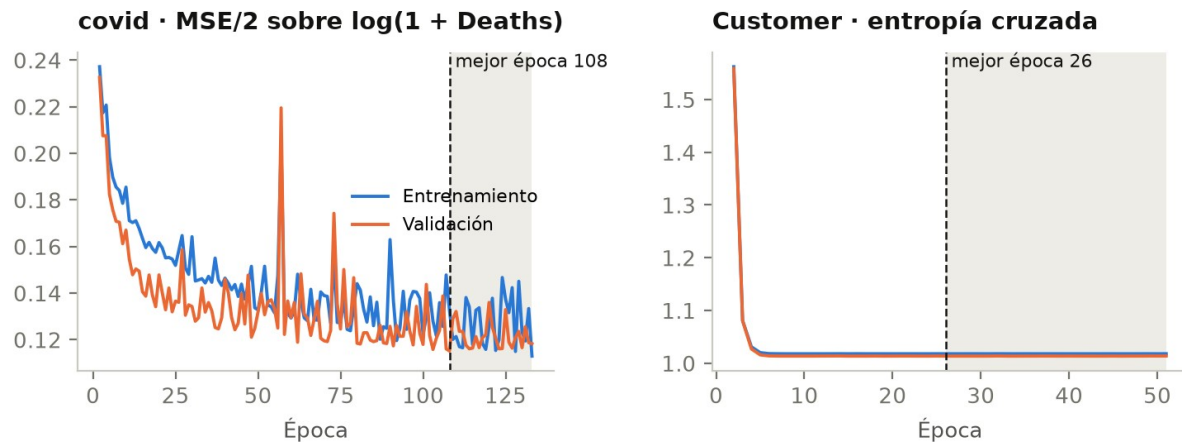


Figura 10. Pérdida por época de las dos redes en entrenamiento y validación, con la mejor época marcada.

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.7.5. Análisis detallado de la red neuronal

Tabla 18. Red de covid: cinco métricas en cada conjunto (escala $\log(1 + Deaths)$)

Métrica	Entrenamiento	Validación	Prueba
R ²	0,9683	0,9721	0,9632
MSE	0,2507	0,2156	0,2896
RMSE	0,5007	0,4643	0,5382
MAE	0,3031	0,3185	0,3326
MedAE	0,1984	0,2116	0,2177

Fuente: Elaboración propia a partir de los resultados del proyecto.

Las cinco métricas son prácticamente iguales en los tres conjuntos: la red generaliza. El RMSE de prueba (0,5382) es cinco veces menor que el del baseline (2,8050). Como el objetivo está en escala logarítmica, el MedAE tiene una lectura directa: $e^{0,2177} = 1,24$, es decir, **la mitad de las predicciones queda a menos de un factor $\times 1,24$ del valor real** (un error menor al 24 %).

La Figura 11 contrasta lo predicho con lo real en el conjunto de prueba. Los puntos siguen la diagonal en todo el rango, sin desviaciones sistemáticas en los valores bajos ni en los altos, y

los residuos se centran en cero (media $-0,008$) con forma simétrica: la red no sobreestima ni subestima de forma sistemática. Los pocos puntos alejados corresponden a registros con cero fallecidos reportados, un caso que ninguna variable de entrada anticipa.

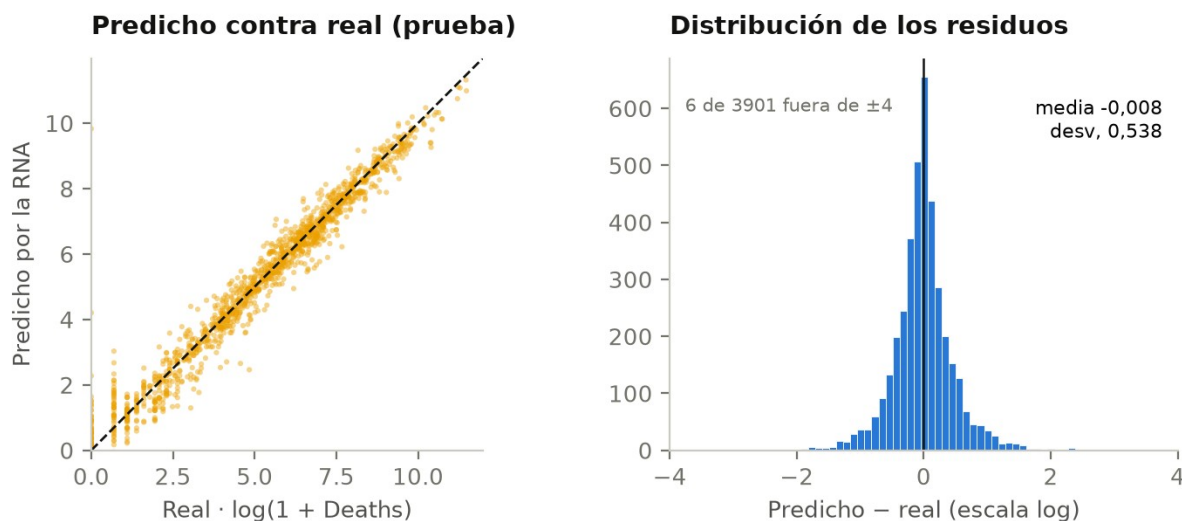


Figura 11. Red de covid: valor predicho contra valor real en prueba (la línea es la predicción perfecta) y distribución de los residuos.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Tabla 19. Red de Customer: cinco métricas en cada conjunto

Métrica	Entrenamiento	Validación	Prueba
Exactitud	0,5123	0,5161	0,5097
Precisión macro	0,1708	0,1720	0,1699
Recall macro	0,3333	0,3333	0,3333
F1 macro	0,2258	0,2270	0,2251
F1 ponderado	0,3471	0,3514	0,3441

Fuente: Elaboración propia a partir de los resultados del proyecto.

La matriz de confusión (Figura 12) confirma que la red asigna todos los clientes de prueba a Consumer. La salida softmax es casi idéntica para cualquier cliente y coincide con la proporción de cada segmento en entrenamiento (51,3 % contra 51,2 %, 30,2 % contra 30,0 % y 18,5 % contra 18,8 %). Es el comportamiento óptimo de un modelo cuando las variables de entrada no contienen información: la mejor estimación de la probabilidad de cada clase es su frecuencia. No es un fallo de la red; es la respuesta correcta a un problema sin señal, y la sección siguiente lo demuestra.

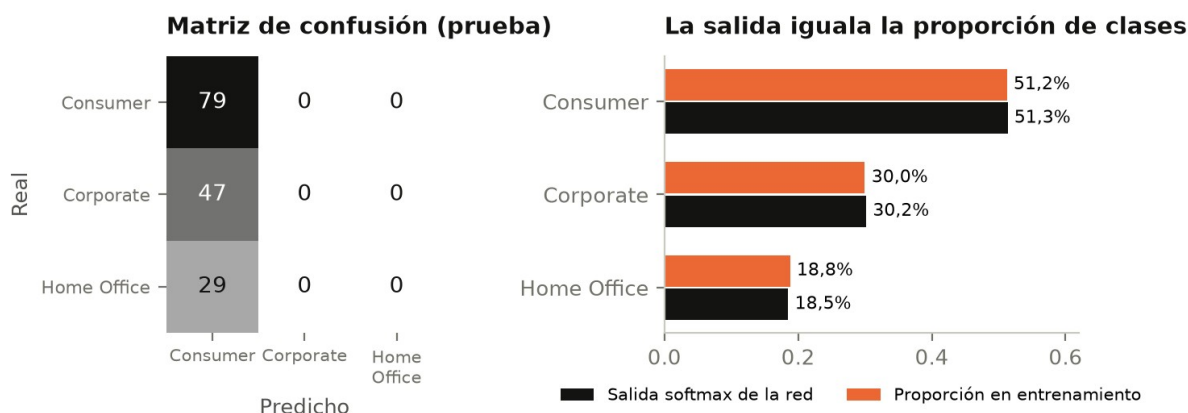


Figura 12. Red de Customer: matriz de confusión en prueba y comparación de su salida con la proporción de clases.

Fuente: Elaboración propia a partir de los resultados del proyecto.

4.7.6. Diagnóstico de Customer.csv: ¿hay señal que aprender?

Para distinguir entre «los modelos fallan» y «no hay nada que aprender» se realizaron cuatro experimentos (src/diagnostico_customer.py), resumidos en la Figura 13.

- **Un árbol sin límite de profundidad se ajusta casi perfecto** —0,9790 en entrenamiento, con 83 niveles y 310 hojas para 620 filas— y cae a 0,4323 en prueba. Memoriza, no aprende. Ni siquiera llega a 1,0000 porque hay clientes idénticos en edad y ubicación con distinto segmento.
- **Buscar hiperparámetros no mueve el techo.** Una búsqueda en malla de 24 combinaciones con validación cruzada de 5 pliegues encuentra como mejor configuración un bosque de profundidad 3 (0,5145 en validación cruzada, 0,5032 en prueba), que en la práctica responde la clase mayoritaria.
- **Prueba de permutación.** El mismo modelo entrenado 30 veces con las etiquetas **barajadas al azar** obtiene $0,4983 \pm 0,0193$, y 25 de las 30 corridas igualan o superan al modelo con las etiquetas reales (0,4903). Si romper la relación entre variables y segmento no empeora el resultado, es porque no había relación.
- **Control positivo.** Con el mismo código, las mismas columnas y el mismo árbol, cambiar el objetivo a Region da exactitud **1,0000**: el estado determina la región. El pipeline funciona; lo que falta es información sobre el segmento.

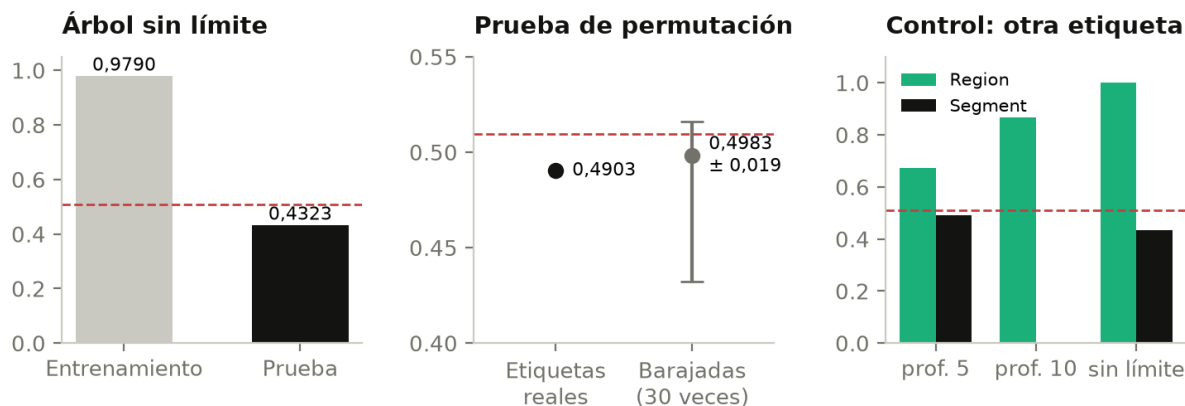


Figura 13. Experimentos de diagnóstico sobre Customer.csv. La línea roja es el baseline.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Además se probaron ocho formas distintas de preparar el mismo archivo (src/enfoques_customer.py), con tres modelos cada una:

Tabla 20. Enfoques alternativos de preparación de Customer.csv (mejor de tres modelos)

Enfoque	Qué cambia	Columnas	Exactitud	Baseline
Base (el del informe)	Edad y ubicación, one-hot	608	0,4903	0,5097
Sin alta cardinalidad	Sin ciudad ni código postal	44	0,5157	0,5157
Solo la edad	Una sola columna	1	0,5157	0,5157
Sin limpieza	Con duplicados, ID y nombre	2.192	0,5157	0,5157
Codificación ordinal	Cada categoría como un número	5	0,5097	0,5097
Edad en tramos	Edad discretizada en 6 rangos	612	0,4968	0,5097
Binario Consumer / Otro	Dos clases en lugar de tres	608	0,4968	0,5097
Objetivo Region (control)	Se cambia la etiqueta	605	1,0000	0,3224

Fuente: Elaboración propia a partir de los resultados del proyecto.

Siete preparaciones distintas, ninguna supera su baseline; varias lo empatan exactamente, la firma de un modelo que responde siempre la clase mayoritaria. El único cambio que produce aprendizaje es cambiar la etiqueta. La conclusión es firme: el resultado no depende de cómo se limpió u organizó el dataset, sino de que la edad y la ubicación no contienen información sobre el segmento. Ese comportamiento vive en el historial de compras (volumen, frecuencia, categorías), que no forma parte de este archivo.

4.7.7. Mejor ajuste sin sobreajuste: modelo elegido

En covid, Random Forest obtiene el mayor R^2 de prueba (0,9686) y la red desde cero el segundo (0,9632), a solo 0,0054. Se eligió **la red neuronal desde cero como modelo final** para las

predicciones del usuario porque ofrece el mejor equilibrio entre ajuste y generalización: su brecha entre entrenamiento y prueba es cinco veces menor (0,0051 contra 0,0273), su entrenamiento está controlado por un conjunto de validación, y su funcionamiento completo — pesos, activaciones y gradientes— es visible y verificable. En Customer.csv ningún modelo es utilizable; la red se conserva en la aplicación para mostrar de forma transparente esa conclusión.

4.8. Predicciones con datos ingresados por el usuario

El usuario puede ingresar sus propios valores de dos formas: en la **página web de la red neuronal**, dentro de la aplicación, o en la **terminal**, ejecutando `red_neuronal.py`. En ambos casos los valores pasan exactamente por la misma transformación que los datos de entrenamiento — $\log(1 + x)$, cálculo del día de la pandemia, codificación one-hot y estandarización con la media y la desviación del entrenamiento— antes de entrar a la red.

4.8.1. Interfaz de predicción

La página ofrece dos tareas. Para no exigir que el usuario conozca el dataset, cada una parte de **casos reales del conjunto de prueba** que se cargan con un clic, y cada campo explica qué valores son razonables (Figura 14). La predicción se recalcula al instante con cada cambio y se acompaña de su interpretación, de la comparación con el valor real cuando se usa un caso del dataset, y de las neuronas que se activan en cada capa.

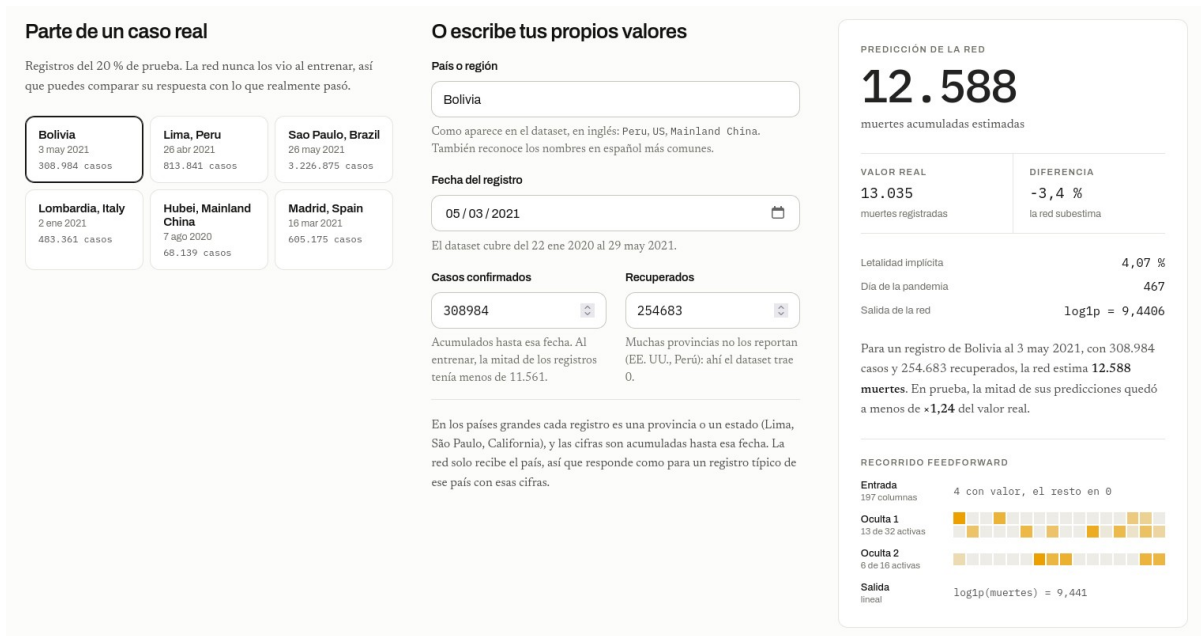


Figura 14. Predicción de fallecidos: casos reales, campos de ingreso y resultado con su interpretación y el recorrido feedforward.

Fuente: Captura de la aplicación del proyecto.

En la tarea de clientes, la ubicación se elige con listas encadenadas —región, estado, ciudad y código postal— que solo ofrecen combinaciones existentes en el dataset (Figura 15). El resultado muestra la probabilidad de cada segmento junto a su proporción en el entrenamiento, y advierte que la red no supera el baseline.

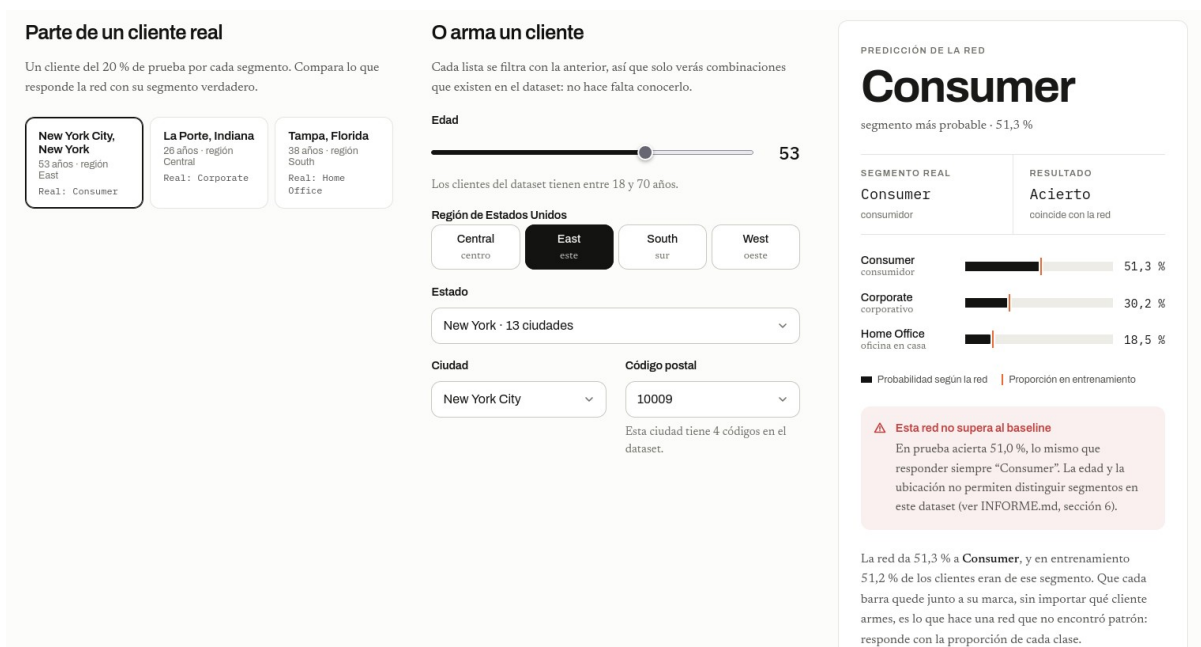


Figura 15. Predicción del segmento de un cliente con listas encadenadas y advertencia sobre el baseline.

Fuente: Captura de la aplicación del proyecto.

Los datos del usuario se validan antes de predecir, porque una red devuelve un número para cualquier entrada, tenga sentido o no:

Tabla 21. Validaciones aplicadas a los datos del usuario

Caso	Respuesta del sistema
País que no está en el dataset	Se rechaza y se sugieren nombres parecidos; se reconocen los nombres en español más comunes («Estados Unidos» → US).
Fecha fuera del período	Se rechaza: el dataset cubre del 22 de enero de 2020 al 29 de mayo de 2021.
Texto o número negativo en un conteo	Se rechaza con un mensaje que indica el formato correcto.
Valor fuera del rango de entrenamiento	Se predice, pero se advierte que es una extrapolación y que la red no es confiable ahí.
Más recuperados que confirmados	Se advierte que esa combinación no existe en datos reales.
Combinación de ubicación inexistente	No se puede elegir: cada lista se filtra con la anterior.

Fuente: Elaboración propia a partir de los resultados del proyecto.

En la terminal, la misma validación funciona así (sesión real):

```
1) Predecir muertes por covid  2) Predecir segmento de cliente  0) Salir
Opcion: 1
Pais o region (ej. Peru, Mexico, Brazil): Peruu
'Peruu' no aparece en los datos de entrenamiento.
Pais o region (ej. Peru, Mexico, Brazil): peru
Fecha de observacion (AAAA-MM-DD): 15/06/2020
Formato invalido; ejemplo: 2020-06-15.
Fecha de observacion (AAAA-MM-DD): 2019-01-01
El dataset empieza el 2020-01-22; ingrese una fecha posterior.
Fecha de observacion (AAAA-MM-DD): 2020-06-15
Casos confirmados acumulados: abc
Debe ser un numero.
Casos confirmados acumulados: -5
Debe ser un numero finito mayor o igual a 0.
Casos confirmados acumulados: 229736
Recuperados acumulados: 116099

Salida de la red: log1p(Deaths) = 9.2614
Muertes acumuladas estimadas: 10,523
Letalidad implicita: 4.58% de los confirmados.
```

4.8.2. Casos de prueba e interpretación: fallecidos por COVID-19

Primero, seis registros del conjunto de prueba que la aplicación ofrece como ejemplos:

Tabla 22. Predicción de la red para registros del conjunto de prueba

Registro	Fecha	Confirmados	Fallecidos reales	Predicción	Diferencia
Bolivia	2021-05-03	308.984	13.035	12.588	-3,4 %
Lima, Peru	2021-04-26	813.841	26.248	24.747	-5,7 %
Sao Paulo, Brazil	2021-05-26	3.226.875	109.241	63.367	-42,0 %
Lombardia, Italy	2021-01-02	483.361	25.281	25.203	-0,3 %
Hubei, Mainland China	2020-08-07	68.139	4.512	3.735	-17,2 %
Madrid, Spain	2021-03-16	605.175	14.354	12.633	-12,0 %

Fuente: Elaboración propia a partir de los resultados del proyecto.

Cinco de los seis casos quedan dentro de un 18 % del valor real, y Lombardía prácticamente exacta. El caso de São Paulo (-42 %) es instructivo: es el registro más grande de Brasil, con más de tres millones de casos, en una zona del espacio con pocos ejemplos de entrenamiento de esa magnitud; la red tiende a subestimar los extremos, como todo modelo que aprende del caso típico.

Luego, seis consultas como las que haría un usuario, con cifras que se buscaron en el dataset para poder contrastar la predicción. Las cuatro primeras corresponden a registros que **no estaban en la muestra** con la que se trabajó: son datos completamente nuevos para la red.

Tabla 23. Predicciones con datos ingresados por el usuario

País	Fecha	Confirmados	Recuperados	Predicción	Real	Dif.
Bolivia	2020-08-01	78.793	23.968	3.019	3.064	-1,5 %
Bolivia	2021-01-15	183.589	142.279	9.473	9.571	-1,0 %
Argentina	2020-10-01	765.002	603.140	17.450	20.288	-14,0 %
Mexico	2020-05-01	20.739	12.377	2.046	1.972	3,8 %
Peru	2020-06-15	232.992	116.099	10.681	6.860	55,7 %
US	2021-05-29	33.251.939	0	611.124	594.306	2,8 %

Fuente: Elaboración propia a partir de los resultados del proyecto.

- **Bolivia, 1 de agosto de 2020.** Con 78.793 casos, la red estima 3.019 fallecidos contra 3.064 reales (-1,5 %). La letalidad implícita de la predicción, 3,83 %, coincide con la real (3,89 %).
- **Bolivia, 15 de enero de 2021.** Con más del doble de casos, la red estima 9.473 contra 9.571 (-1,0 %). La letalidad implícita sube a 5,16 %, igual que la real (5,21 %): la red reproduce el aumento de la letalidad acumulada entre ambas fechas, que no se deduce solo de los casos.

- **Argentina y México.** Errores de -14,0 % y 3,8 %, dentro del margen típico del modelo. En México, con pocos casos y alta letalidad (9,5 %), la red reproduce correctamente la alta proporción de fallecidos del país.
- **Perú, total nacional.** La red sobreestima (55,7 %). En esa fecha Perú reportaba por región, y la red aprendió de registros regionales; ingresar la suma nacional es pedirle una combinación que no vio. Muestra la importancia de ingresar datos del mismo tipo que los de entrenamiento.
- **Estados Unidos, total nacional.** 33.251.939 casos superan el máximo del entrenamiento (5.698.706); la aplicación advierte que es una extrapolación. Aun así, la estimación (611.124) queda a 2,8 % del valor real. Es un acierto que no debe generalizarse: fuera del rango de entrenamiento el modelo no ofrece garantías, y por eso la aplicación lo advierte.

4.8.3. Casos de prueba e interpretación: segmento de un cliente

Tabla 24. Predicciones de la red de Customer para clientes ingresados por el usuario

Edad	Ubicación	Consumer	Corporate	Home Office
25	Los Angeles, California (90036, West)	51,3 %	30,2 %	18,5 %
67	Henderson, Kentucky (42420, South)	51,3 %	30,2 %	18,5 %
45	New York City, New York (10009, East)	51,3 %	30,2 %	18,5 %
19	Houston, Texas (77095, Central)	51,3 %	30,2 %	18,5 %

Fuente: Elaboración propia a partir de los resultados del proyecto.

Cuatro clientes muy distintos —de 19 a 67 años, en las cuatro regiones del país— reciben exactamente las mismas probabilidades, iguales a la proporción de cada segmento en el entrenamiento. La interpretación correcta es que **con estas variables no es posible predecir el segmento**: la red responde «Consumer» porque es la clase más frecuente, no porque haya reconocido un patrón en el cliente. Presentar esto con honestidad —la aplicación lo advierte en cada predicción— es preferible a ofrecer una predicción que aparenta una certeza que el dataset no permite.

4.9. Aplicación integrada

Todo el sistema se ejecuta con un solo comando, `npm run dev` (o `.venv/bin/python src/servidor.py`, sin Node). El comando inicia un servidor local y abre el navegador en una aplicación con tres vistas en un menú lateral:

- **Entrenar:** ejecuta los seis scripts en orden —o uno solo— y muestra el estado de cada paso, su tiempo y la salida de la consola en vivo. Al terminar, las otras dos vistas se actualizan con los resultados nuevos (Figura 16).
- **Ocho modelos:** el dashboard con la comparación de modelos, la trazabilidad de la limpieza —incluida la huella SHA-256 de cada archivo leído, que permite verificar que los resultados salieron de esos datos exactos— y el diagnóstico.
- **Red neuronal:** la página de predicción de la sección 4.8, con la validación de la red.

Machine learning
Aprendizaje supervisado sobre covid y clientes

▶ **Entrenar**
Correr el pipeline completo

⚙️ **Ocho modelos**
Comparación, limpieza y diagnóstico

🧠 **Red neuronal**
Predice con tus propios valores

Entrenar los modelos

Corre los seis scripts del proyecto en orden: limpieza y los ocho modelos, el diagnóstico de Customer.csv, las funciones de los apuntes, la red neuronal y el dashboard. Al terminar, las vistas **Ocho modelos** y **Red neuronal** muestran los resultados nuevos.

Entrenar todo = 5 min 7 s

✅ Entrenamiento terminado en 50 s. Los resultados ya están actualizados. [Ver los ocho modelos](#) [Probar la red neuronal](#)

☐ **Ocho modelos sobre los dos datasets** = 3 min 20 s

src/aprendizaje_supervisado.py - resultados_covid.csv, resultados_customer.csv, resumen.json, limpieza.json
Última vez: 2026-09-29 00:35

✅ **Diagnóstico de Customer.csv** 21 s

src/diagnostico_customer.py - diagnostico_customer.txt
Listo en 21 s

✅ **Ocho formas de preparar Customer.csv** 10 s

src/enfoques_customer.py - enfoques_customer.csv, enfoques_customer.txt
Listo en 10 s

✅ **Funciones de los apuntes** 2 s

src/funciones.py - funciones.json
Listo en 2 s

✅ **Red neuronal desde cero** 17 s

src/red_neuronal.py - red_neuronal.html, red_neuronal.json
Listo en 17 s

● Resultados al día
Entrenado en 50 s

Salida en vivo

```

Entrenamiento  Validacion  Prueba
R2              0.9683      0.9721  0.9632
MSE             0.2507      0.2156  0.2896
RMSE            0.5007      0.4643  0.5382
MAE             0.3031      0.3185  0.3326
MedAE           0.1984      0.2116  0.2177
Baseline (predecir siempre la media): RMSE 2.8059
Brecha de R2 entrenamiento - prueba: 0.8851 (sin sobreajuste apreciable)

=====
RNA 2. Customer.csv -> clasificacion de Segment
=====
Arquitectura: 608 -> 16 -> 3 neuronas (ReLU en ocultas, softmax en salida)
Division: entrenamiento 527 / validacion 93 / prueba 155
Tasa 0.01, momento 0.9, L2 1.0
Early stopping: mejor epoca 26 de 51 corridas (perdida validacion 1.0134)

Metricas:
Entrenamiento  Validacion  Prueba
Exactitud      0.5123      0.5161  0.5097
Precision macro 0.1788      0.1720  0.1699
Recall macro   0.3333      0.3333  0.3333
F1 macro       0.2258      0.2270  0.2251
F1 ponderado   0.3471      0.3514  0.3441

Matriz de confusion (prueba):
pred Consumer  pred Corporate  pred Home Office
real Consumer   79              0              0
real Corporate  47              0              0
real Home Office 29              0              0
Baseline (predecir siempre 'Consumer'): exactitud 0.5097 -> la red NO SUPERA al baseline

```

Figura 16. Vista Entrenar de la aplicación después de ejecutar cuatro pasos, con la salida real de los scripts.

Fuente: Captura de la aplicación del proyecto.

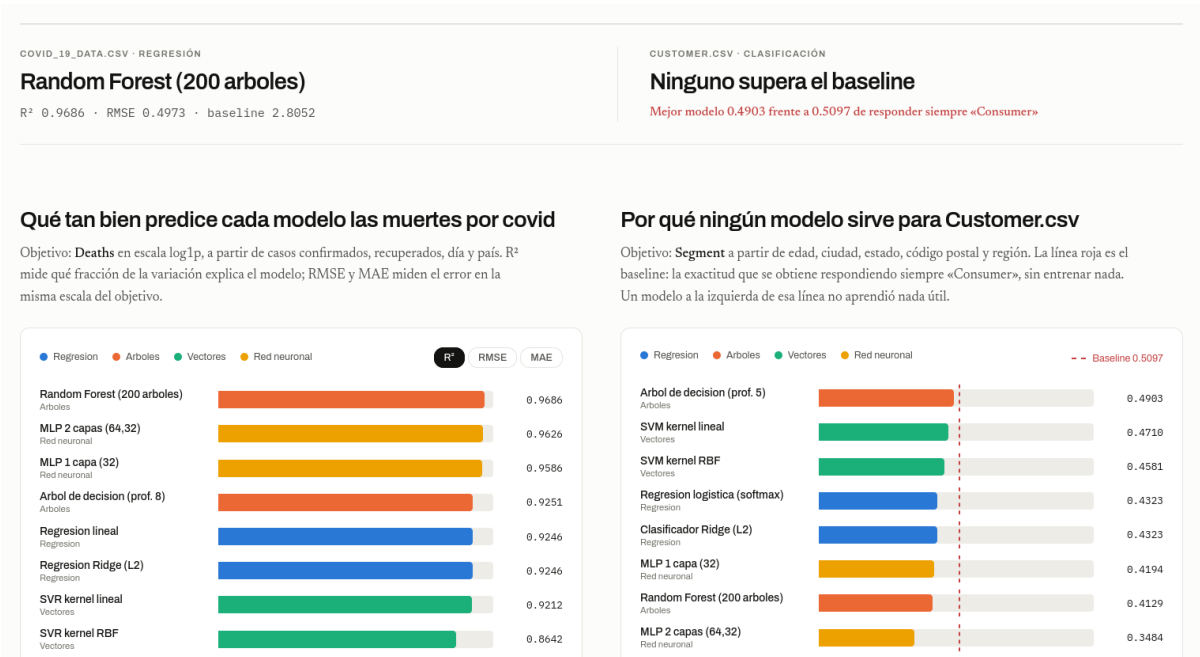


Figura 17. Vista Ocho modelos: comparación de los modelos en ambos datasets.
Fuente: Captura de la aplicación del proyecto.

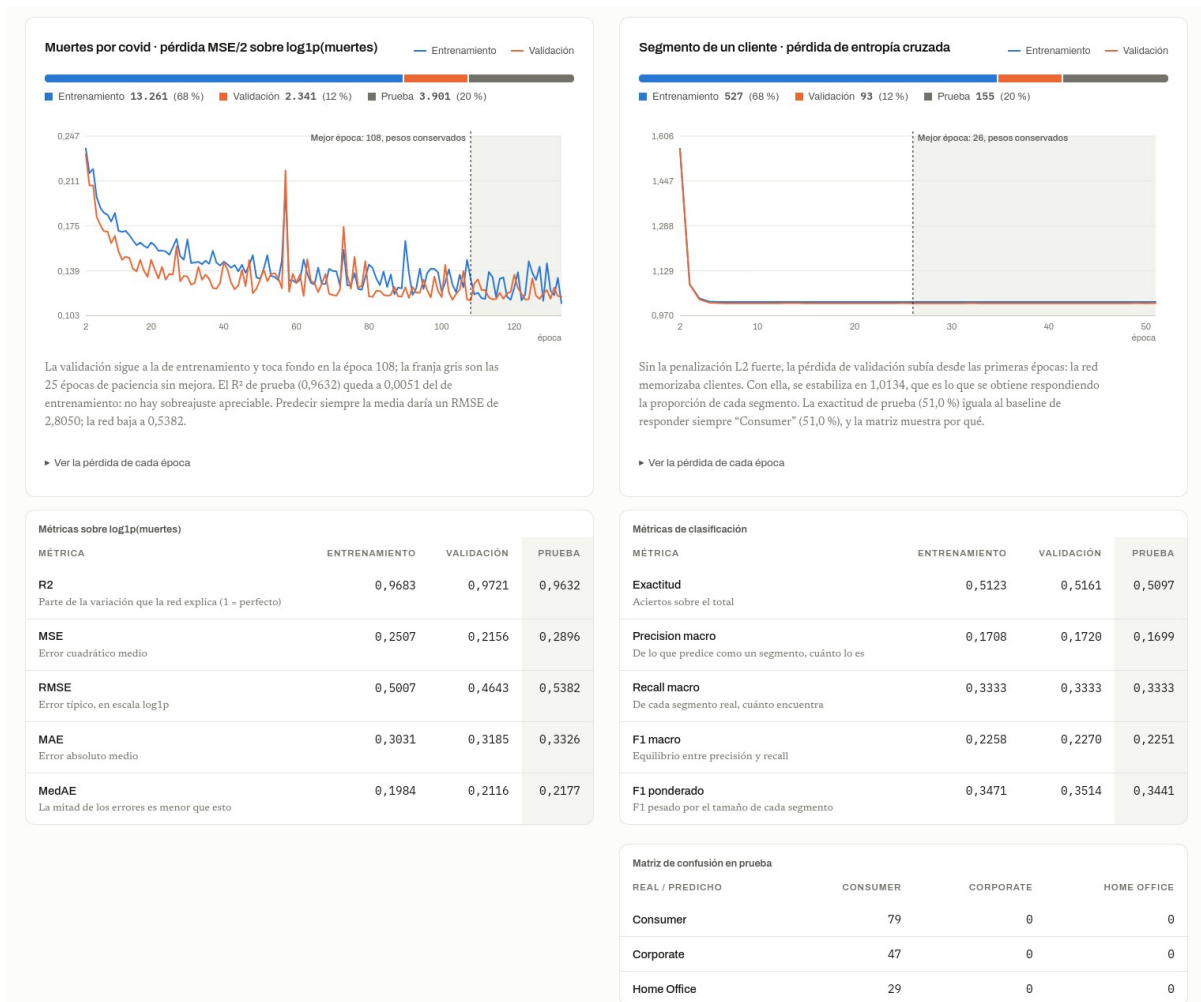


Figura 18. Sección de validación de la red dentro de la aplicación: partición, curvas de pérdida, métricas y matriz de confusión.

Fuente: Captura de la aplicación del proyecto.

El servidor solo acepta conexiones desde la propia computadora y solo ejecuta los scripts del proyecto. Si se recarga la página durante un entrenamiento, este continúa y su registro se vuelve a mostrar; también puede cancelarse.

V. Conclusiones

- **Pre-procesamiento.** Cada técnica se eligió según la naturaleza del dato: la columna de provincia se eliminó en lugar de imputarse porque su ausencia es estructural; la transformación $\log(1 + x)$ antes del filtro IQR evitó descartar como atípicos a los países grandes; y los identificadores se excluyeron antes de codificar. Todo el proceso es código propio, reproducible y trazable.
- **Covid: el problema tiene solución y la red neuronal la alcanza.** Los nueve modelos superan con amplitud al baseline. La red neuronal desde cero obtiene un R^2 de 0,9632 en prueba, a 0,0054 de Random Forest, con una brecha entre entrenamiento y prueba de solo 0,0051, cinco veces menor que la del bosque; la mitad de sus predicciones queda a menos de un factor $\times 1,24$ del valor real.
- **La implementación propia es correcta.** El feedforward, la retropropagación y el descenso de gradiente con momento se verificaron con el gradiente numérico, con el problema XOR y por su equivalencia con el MLP de scikit-learn. El early stopping sobre un conjunto de validación separado controló el sobreajuste sin tocar el conjunto de prueba.
- **Customer: el problema no tiene solución con estas variables, y eso es un resultado.** Ningún modelo supera el baseline. La prueba de permutación, el control con Region y las ocho preparaciones alternativas demuestran que la edad y la ubicación no contienen información sobre el segmento. La red, bien regularizada, responde la proporción de clases: el comportamiento óptimo sin señal. Predecir el segmento requeriría variables de comportamiento de compra.
- **Predicciones del usuario.** Con datos nuevos, la red estimó los fallecidos de Bolivia, Argentina y México con errores entre el 1 % y el 14 %. Los casos que se apartan —un total nacional de un país que reporta por región, un valor fuera del rango de entrenamiento— muestran por qué la aplicación valida y advierte sobre los datos ingresados.
- **Aplicación.** El entrenamiento, los resultados y la predicción quedan integrados en una aplicación que se ejecuta con un solo comando, y cada cifra de las páginas se genera desde los resultados de los scripts.

Limitaciones y trabajo futuro

- El modelo de covid estima los fallecidos del mismo registro; no pronostica. Un pronóstico requeriría modelar la serie temporal con datos de días anteriores.

- Se usó una sola partición 80/20; una validación cruzada daría intervalos de confianza para cada métrica.
- La muestra de 20.000 registros responde a una restricción de tiempo de cómputo; la constante COVID_MUESTRA permite ampliarla.
- Para Customer.csv, el siguiente paso sería unir el historial de órdenes por Customer ID, donde vive la información de comportamiento de compra.

VI. Bibliografía

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Cortes, C. y Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Dong, E., Du, H. y Gardner, L. (2020). An interactive web-based dashboard to track COVID-19 in real time. *The Lancet Infectious Diseases*, 20(5), 533–534.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3.^a ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y. y Courville, A. (2016). *Deep Learning*. MIT Press.
- Han, J., Kamber, M. y Pei, J. (2012). *Data Mining: Concepts and Techniques* (3.^a ed.). Morgan Kaufmann.
- Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hastie, T., Tibshirani, R. y Friedman, J. (2009). *The Elements of Statistical Learning* (2.^a ed.). Springer.
- He, K., Zhang, X., Ren, S. y Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *Proceedings of the IEEE International Conference on Computer Vision*, 1026–1034.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prechelt, L. (1998). Early stopping — but when? En G. Orr y K.-R. Müller (Eds.), *Neural Networks: Tricks of the Trade* (pp. 55–69). Springer.
- Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.

Anexo. Estructura del proyecto y ejecución

El código fuente completo se entrega en formato digital junto a este informe. Su estructura es la siguiente:

Tabla 25. Estructura del proyecto

Ruta	Contenido
datos/	Customer.csv y covid.zip (se descomprime automáticamente en datos/covid/).
src/	Los scripts de la Tabla 3, la plantilla de la página de la red y la aplicación (servidor.py, app.html).
resultados/	Todas las salidas: tablas de métricas, trazabilidad de la limpieza, registros de ejecución y páginas web.
docs/	Informes de los avances y material de la materia.
package.json	Define los comandos npm run dev y npm run setup.
README.md	Instrucciones de instalación y ejecución.

Fuente: Elaboración propia a partir de los resultados del proyecto.

Para ejecutarlo desde cero:

```
# 1. Crear el entorno (una sola vez; requiere uv)
npm run setup
#   equivalente: uv venv --python 3.12 .venv
#               uv pip install --python .venv/bin/python pandas scikit-learn

# 2. Abrir la aplicación (entrenar, ver resultados y predecir)
npm run dev
#   equivalente sin Node: .venv/bin/python src/servidor.py

# Verificar la retropropagación contra el gradiente numérico
.venv/bin/python src/test_red_neuronal.py

# Predicción en la terminal
.venv/bin/python src/red_neuronal.py
```